



RAPPORT DE PROJET

ECUE SAE_J3D : Projet d'étude - Développement d'un jeu vidéo

Deadly Inspection

Conception et réalisation d'un jeu d'enquête dans un univers
post-apocalyptique

VALLIKANTHAN Rakavi
MOUSSAOUI Jasmine
ROBBE Gabriel
KARREY Amine
JOURNÉ Jean-Baptiste

Année universitaire 2025 - 2026





Sommaire :

1) Introduction	1
2) Présentation et naissance du projet	3
2.1) Origine du concept	3
2.2) Présentation générale de deadly inspection	4
2.3) Le Gameplay asymétrique : Bob et Eliott	6
2.4) Inspirations et influences du projet	7
2.5) Les ambitions initiales du projet	8
2.6) Les ambitions initiales du projet	10
3) Première phase de développement : l'architecture 3D	11
3.1) Choix de l'architecture technique initiale	11
3.2) Premiers prototypes et expérimentations techniques	13
3.3) Développement du système multijoueur et communication réseau	14
3.4) Génération environnement 3D	16
4) Les limites du prototypes 3D	18
4.1) Une complexité technique plus importante que prévue	18
4.2) Les difficultés d'intégration et la fragmentation du projet	19
4.3) Contrainte temporelles et remise en question du projet	21
4.4) Abandon de la 3D et transition vers la 2D	23
5) Reconstruction complète du projet en 2D	24
5.1) Reconstruction technique nécessaire	24
5.2) Nouvelle direction artistique et identité visuelle	26
5.3) Conception de la carte et utilisation de Tiled	27
5.4) Développement des interactions et des systèmes de gameplay	29
5.5) Développement des interactions et des systèmes de gameplay	30
5.6) Système d'inspection, quarantaine et Intelligence Artificielle	32
6) Développement du multijoueur et synchronisation réseau	34
6.1) Objectif du multijoueur	34
6.2) Premières expérimentations réseau et architecture client/serveur	36
6.3) Reconstruction du système réseau et nouvelles approches de synchronisation	37
6.4) Synchronisation des survivants et gestion de la file d'attente	39
6.5) Travail collaboratif, Git et gestion des conflits technique	41
6.6) Ce que le développement multijoueur nous as appris	43
7.0) Immersion et sonore	44
7.1) Cycle jour/nuit	44
7.2) Sons d'ambiance et effet sonore	46
7.3) Le menu principale	47
8.0) Améliorations	48
9.0) Conclusion	49



1) Introduction

Dans le cadre de notre première année à l'EPITA, école d'ingénieurs spécialisée dans l'informatique et les technologies du numérique, il nous a été demandé de réaliser un projet de développement informatique en groupe sur une durée d'environ huit mois. Ce projet constitue une étape importante de la formation, puisqu'il permet de mettre en application de manière concrète les différentes notions abordées durant les cours et travaux pratiques, aussi bien sur le plan technique que méthodologique.

Contrairement aux exercices académiques réalisés durant l'année, ce projet s'inscrit dans une logique beaucoup plus proche d'un véritable développement logiciel. Il ne s'agissait pas uniquement de produire du code fonctionnel, mais également de réfléchir à l'organisation du travail, à la conception d'une architecture cohérente, à la gestion des contraintes techniques ainsi qu'à la coordination entre plusieurs développeurs travaillant simultanément sur un même projet.

Le sujet imposé consistait à développer un jeu vidéo en Python intégrant obligatoirement plusieurs éléments techniques, notamment une composante multijoueur réseau ainsi qu'une forme d'intelligence artificielle. Ces contraintes représentaient un défi important dans le cadre d'un projet étudiant, en particulier pour une équipe découvrant pour la première fois certaines problématiques liées au développement de jeux vidéo et à la programmation réseau.

C'est dans ce contexte qu'est né *Deadly Inspection*, un jeu vidéo coopératif se déroulant dans un univers post-apocalyptique. Le projet repose sur une idée centrale : placer deux joueurs dans des rôles complémentaires au sein d'un refuge représentant l'un des derniers bastions encore fonctionnels de l'humanité après une pandémie ayant transformé une partie de la population en créatures hostiles.

Le gameplay du jeu repose sur une coopération asymétrique entre deux personnages possédant des responsabilités totalement différentes. Le premier joueur incarne Bob, chargé de surveiller l'entrée du refuge et d'inspecter les survivants souhaitant rejoindre la base. Son rôle est d'identifier les individus potentiellement infectés à partir de différents indices physiques ou comportementaux. Le second joueur incarne Eliott, un scientifique responsable de la gestion interne du refuge. Il doit gérer les ressources, organiser les survivants, développer de nouveaux outils et réagir aux crises provoquées par les erreurs d'inspection ou les événements extérieurs.

À travers cette asymétrie de gameplay, notre objectif était de créer une expérience coopérative reposant davantage sur la communication, l'anticipation et la prise de décision que sur l'action pure. Nous souhaitions développer un jeu où les conséquences des choix des joueurs auraient un impact direct sur l'évolution de la partie et sur la stabilité du refuge.



Au lancement du projet, notre ambition technique était particulièrement importante. Nous avons initialement choisi de développer Deadly Inspection dans un environnement 3D à l'aide du moteur Panda3D. Ce choix était motivé par notre volonté de proposer une expérience immersive et visuellement plus ambitieuse. L'utilisation de la 3D devait permettre de renforcer l'atmosphère du jeu, de rendre l'environnement plus vivant et d'offrir davantage de liberté dans la conception des interactions et du gameplay.

Cependant, au fil du développement, cette première architecture s'est révélée beaucoup plus complexe à maîtriser que prévu. Les problématiques liées à l'intégration des différents systèmes, à la synchronisation réseau, à la gestion des animations, au rendu graphique et à la stabilité globale du projet ont progressivement ralenti l'avancement du développement. Malgré plusieurs mois de travail, le projet restait fragmenté en différents prototypes techniques difficiles à réunir dans une version réellement jouable.

Cette situation nous a amenés à prendre une décision majeure dans le développement du projet : abandonner entièrement l'architecture 3D afin de reconstruire le jeu dans un environnement 2D à l'aide de Pygame. Bien qu'importante, cette transition s'est révélée déterminante pour la suite du projet. Elle nous a permis de simplifier de nombreux aspects techniques, de restructurer notre développement et surtout de transformer progressivement notre prototype en un véritable jeu fonctionnel.

Au-delà de la simple réalisation technique, ce projet nous a également permis de découvrir les réalités du travail en équipe dans un contexte de développement logiciel. La répartition des tâches, la communication entre les membres, la gestion des conflits techniques, les difficultés liées aux merges Git ou encore l'adaptation constante du planning ont occupé une place centrale tout au long du projet.

Ce rapport retrace l'ensemble de cette évolution. Il présente les différentes étapes du développement de Deadly Inspection, depuis les premières ambitions du projet jusqu'à la reconstruction complète du jeu en 2D. Il détaille les choix techniques réalisés, les difficultés rencontrées, les mécaniques de gameplay développées ainsi que les enseignements tirés de cette expérience, aussi bien sur le plan humain que technique.

Dans une première partie, nous présenterons la naissance du projet ainsi que les ambitions initiales ayant conduit au choix d'une architecture 3D. Nous aborderons ensuite les différentes problématiques rencontrées durant cette première phase de développement, notamment les difficultés liées à l'intégration des systèmes, au multijoueur et à la stabilité du projet.



Dans un second temps, nous expliquerons les raisons ayant conduit à la transition complète du projet vers une architecture 2D basée sur Pygame, ainsi que les bénéfices apportés par ce changement.

Le rapport détaillera également les principales mécaniques de gameplay développées pour *Deadly Inspection*, notamment le système de coopération asymétrique entre Bob et Eliott, la gestion du refuge et le système d'analyse des survivants.

Enfin, nous présenterons les enseignements tirés de cette expérience, les difficultés rencontrées durant le développement ainsi que les perspectives d'évolution envisagées pour la suite du projet.

2) Présentation et naissance du projet

2.1) Origine du concept

Le projet *Deadly Inspection* est né d'une volonté commune de concevoir une expérience multijoueur originale reposant davantage sur la coopération et la prise de décision que sur l'action classique présente dans de nombreux jeux vidéo de survie. Dès les premières phases de réflexion, notre objectif était de développer un jeu capable de créer une tension constante entre les joueurs, non pas à travers des combats permanents, mais grâce aux conséquences des choix effectués au cours de la partie.

L'idée d'un univers post-apocalyptique s'est rapidement imposée au sein du groupe. Ce type d'environnement permettait de créer un contexte crédible dans lequel chaque décision prise par le joueur pouvait avoir un impact direct sur la survie de la communauté. L'univers zombie présentait également un avantage important sur le plan du gameplay : il introduisait naturellement une notion de suspicion, de danger permanent et de gestion de crise, éléments qui correspondaient parfaitement à l'expérience que nous souhaitions proposer.

Très tôt dans le développement du concept, nous avons cherché à éviter un gameplay trop classique basé uniquement sur le combat ou l'exploration. Nous voulions que les joueurs soient constamment amenés à communiquer, à coopérer et à gérer des situations stressantes. C'est cette réflexion qui a progressivement conduit à la création du système de coopération asymétrique entre les deux personnages principaux du jeu.

Le concept du refuge est ensuite apparu comme le cœur central du projet. Plutôt que de faire évoluer les joueurs dans un monde totalement ouvert, nous avons choisi de concentrer l'action autour d'une base représentant le dernier refuge encore fonctionnel de l'humanité. Cette approche permettait de créer un environnement plus contrôlé, mais surtout de placer les



joueurs face à des décisions ayant des conséquences directes sur l'évolution de la communauté.

L'idée principale du gameplay repose alors sur un dilemme permanent : accueillir de nouveaux survivants afin de renforcer le refuge, tout en évitant l'introduction d'individus infectés capables de provoquer une catastrophe à l'intérieur même de la base. Ce système crée une tension psychologique importante, puisque chaque erreur peut avoir des conséquences différées et parfois irréversibles.

Au-delà de l'aspect purement narratif, ce concept présentait également un intérêt technique important. Il permettait d'intégrer naturellement plusieurs mécaniques variées, comme la gestion des ressources, les interactions entre joueurs, les systèmes de décisions, le multijoueur coopératif ainsi que différentes formes d'événements dynamiques. Dès cette phase de conception, nous avons compris que le projet aurait une dimension beaucoup plus ambitieuse qu'un simple jeu multijoueur classique.

Enfin, le choix d'un univers post-apocalyptique nous offrait une grande liberté sur le plan artistique et atmosphérique. Les environnements détruits, les bâtiments abandonnés, les survivants isolés et les situations de crise permanentes permettaient de construire une ambiance immersive renforçant la pression ressentie par les joueurs tout au long de la partie.

2.2) Présentation générale de deadly inspection

Deadly Inspection est un jeu vidéo coopératif développé en Python dans le cadre de notre projet annuel à l'EPITA. Le jeu plonge les joueurs dans un univers post-apocalyptique où une infection a provoqué l'effondrement progressif de la société. Au milieu de ce chaos subsiste une unique base de survivants, représentant l'un des derniers espoirs de l'humanité.

Le principe fondamental du jeu repose sur une coopération asymétrique entre deux joueurs possédant des rôles totalement différents mais interdépendants. Contrairement à de nombreux jeux coopératifs où les joueurs effectuent globalement les mêmes actions, *Deadly Inspection* cherche à créer une véritable dépendance stratégique entre les deux personnages.

Le premier joueur incarne Bob, ancien agent de sécurité devenu gardien de l'entrée du refuge. Son rôle consiste à contrôler les survivants souhaitant rejoindre la base. Chaque individu doit être inspecté afin de déterminer s'il représente une menace potentielle pour la communauté. Pour cela, Bob doit observer différents indices visuels ou comportementaux : blessures suspectes, traces de morsures, tremblements, comportements anormaux ou signes physiques liés à l'infection.

Le second joueur incarne Eliott, scientifique et responsable de la gestion interne du refuge. Contrairement à Bob, qui agit principalement à l'extérieur de la base, Eliott évolue à



l'intérieur du refuge et doit gérer les ressources, organiser les survivants, développer de nouveaux outils et réagir aux différentes crises pouvant apparaître au cours de la partie.

Cette séparation des rôles constitue l'élément central du gameplay. Les décisions prises par Bob influencent directement les conditions de survie du refuge, tandis que les actions d'Eliott déterminent la capacité de la base à résister aux différentes menaces. Une erreur de jugement lors d'une inspection peut provoquer plusieurs cycles plus tard une crise majeure à l'intérieur du refuge, obligeant les deux joueurs à collaborer rapidement afin de limiter les dégâts.

L'expérience de jeu repose donc principalement sur la communication, l'anticipation et la gestion des conséquences. Le jeu ne cherche pas uniquement à créer de l'action immédiate, mais plutôt une tension progressive où chaque décision possède un poids important sur l'évolution de la partie.

Le système de progression du jeu est basé sur un cycle de gestion continue. Les survivants acceptés dans la base peuvent apporter de nouvelles compétences utiles au refuge, comme des connaissances médicales, scientifiques ou techniques. Cependant, accepter un individu contaminé peut mettre en danger l'ensemble de la communauté. À l'inverse, refuser un survivant compétent peut ralentir le développement du refuge et compliquer la survie des habitants.

L'ambiance générale du jeu s'inspire fortement des univers post-apocalyptiques présents dans certaines œuvres cinématographiques et vidéoludiques. Nous avons cherché à construire une atmosphère sombre, oppressante et immersive, dans laquelle le joueur ressent en permanence la fragilité de la situation. Les choix artistiques, sonores et narratifs ont été pensés dans cette direction afin de renforcer le sentiment de tension permanent.

Le projet possède également une importante dimension technique. En plus du gameplay coopératif, le développement du jeu nécessite la mise en place de plusieurs systèmes complexes, notamment une architecture multijoueur, la synchronisation réseau entre les joueurs, la gestion des interactions avec l'environnement, le développement d'interfaces distinctes selon le rôle du joueur ainsi qu'une organisation cohérente des différents systèmes composant le jeu.

Enfin, *Deadly Inspection* a été conçu dès le départ comme un projet évolutif. Même si notre objectif principal était avant tout de développer une version stable et jouable dans le cadre de cette soutenance, l'architecture du projet a été pensée de manière suffisamment flexible pour permettre l'ajout futur de nouvelles mécaniques, de nouveaux événements, d'autres rôles jouables ou encore des systèmes de gestion plus avancés.

2.3) Le Gameplay asymétrique : Bob et Eliott

L'un des éléments centraux de *Deadly Inspection* repose sur son système de coopération asymétrique. Dès les premières réflexions autour du projet, nous souhaitions éviter un modèle coopératif classique dans lequel les joueurs effectuent globalement les mêmes actions avec simplement des équipements ou des statistiques différentes. Notre objectif était au contraire de créer une véritable interdépendance entre les joueurs, où chacun possède un rôle unique influençant directement l'expérience de l'autre.

Cette réflexion a conduit à la création des deux personnages principaux du jeu : Bob et Eliott. Bien qu'ils évoluent dans le même univers et poursuivent le même objectif global, assurer la survie du refuge, leurs responsabilités, leurs mécaniques de gameplay et leur manière d'interagir avec le jeu sont totalement différentes.

Bob représente la première ligne de défense du refuge. Son rôle consiste à surveiller l'entrée de la base et à inspecter les survivants souhaitant rejoindre la communauté. Cette mécanique s'inspire volontairement des jeux de prise de décision et d'observation, dans lesquels le joueur doit analyser rapidement des informations parfois contradictoires. Bob doit observer les survivants, détecter des comportements suspects et décider s'ils représentent un danger potentiel.

L'objectif n'était pas simplement de créer un système de sélection binaire entre "infecté" et "non infecté", mais plutôt de générer un doute permanent chez le joueur. Certains survivants présentent des signes évidents d'infection, tandis que d'autres peuvent sembler totalement sains malgré la présence d'indices plus discrets. À l'inverse, certains comportements suspects peuvent simplement être liés à la fatigue, à la peur ou aux blessures provoquées par l'environnement extérieur.

Cette incertitude constitue l'un des principaux éléments de tension du gameplay. Bob doit constamment prendre des décisions rapides tout en sachant qu'une erreur peut avoir des conséquences importantes plusieurs minutes plus tard. Accepter un survivant infecté peut provoquer une contamination progressive à l'intérieur du refuge, tandis que refuser un individu compétent peut ralentir le développement de la base et compliquer la survie de la communauté.

À l'opposé, Eliott possède un gameplay davantage orienté vers la gestion et l'organisation du refuge. Son rôle est moins basé sur l'observation immédiate et davantage sur l'anticipation et la gestion des conséquences. Eliott doit superviser les ressources disponibles, organiser les survivants admis dans la base et développer différents outils permettant d'améliorer les capacités du refuge.

Le laboratoire représente notamment l'un des éléments centraux du rôle d'Eliott. Grâce à celui-ci, il peut développer de nouveaux équipements, améliorer les capacités d'analyse de



Bob ou encore chercher des solutions permettant de ralentir la propagation de l'infection. Cette mécanique crée une véritable boucle de coopération entre les deux joueurs : Bob protège l'entrée du refuge afin de limiter les risques, tandis qu'Eliott améliore progressivement les outils disponibles pour rendre cette tâche plus efficace.

L'interdépendance entre les deux rôles constitue le cœur même du projet. Aucune des deux parties du gameplay ne peut fonctionner correctement de manière isolée. Si Bob refuse trop de survivants, Eliott manque rapidement de ressources et de personnel pour maintenir le refuge opérationnel. À l'inverse, si Eliott gère mal les ressources ou tarde à développer certains outils, Bob se retrouve confronté à des situations de plus en plus difficiles à gérer.

Nous voulions également que cette coopération repose davantage sur la communication humaine que sur des mécaniques automatiques. Les deux joueurs doivent constamment échanger des informations, prévenir les risques et coordonner leurs décisions afin d'assurer la stabilité du refuge. Cette approche permet de créer une expérience plus sociale et plus immersive, dans laquelle la réussite dépend autant de la communication entre les joueurs que de leur maîtrise individuelle des mécaniques du jeu.

Enfin, cette asymétrie de gameplay présente également un intérêt important en termes de rejouabilité. Les deux rôles proposent des expériences très différentes, ce qui encourage les joueurs à changer régulièrement de personnage afin de découvrir une autre facette du jeu. Cette variété permet d'éviter une sensation de répétition et renforce l'identité coopérative de *Deadly Inspection*.

2.4) Inspirations et influences du projet

La conception de *Deadly Inspection* s'est appuyée sur plusieurs inspirations issues à la fois du jeu vidéo, du cinéma et des séries post-apocalyptiques. Ces références nous ont permis de construire progressivement l'univers du jeu, ses mécaniques principales ainsi que son identité visuelle et narrative.

L'une des principales inspirations du projet provient des œuvres post-apocalyptiques mettant l'accent sur la survie humaine et les conséquences psychologiques des situations de crise. Des séries comme *The Walking Dead* ou des films comme *World War Z* et *28 Days Later* ont fortement influencé notre vision de l'univers du jeu. Ces œuvres ne se concentrent pas uniquement sur les créatures hostiles, mais surtout sur les comportements humains, la méfiance, la survie collective et les choix difficiles imposés aux personnages. Cette approche correspondait parfaitement au type d'expérience que nous souhaitions développer.

L'inspiration de *The Last of Us* a également joué un rôle important dans la construction de l'ambiance générale du projet. Bien que notre jeu possède une direction artistique différente,



nous avons été influencés par la manière dont cet univers mélange tension permanente, environnement détruit et narration implicite à travers les décors. L'idée d'un monde progressivement abandonné par la civilisation, où les survivants tentent malgré tout de reconstruire une forme d'organisation, a directement influencé la conception du refuge de *Deadly Inspection*.

Concernant le gameplay, plusieurs jeux coopératifs et de gestion ont également servi de références. Nous nous sommes notamment intéressés aux mécaniques de prise de décision présentes dans certains jeux indépendants, où le joueur doit constamment arbitrer entre sécurité, ressources et survie. Le principe d'inspection des survivants s'inspire indirectement de jeux reposant sur l'analyse et la prise de décision sous pression, dans lesquels chaque choix peut produire des conséquences importantes plusieurs minutes ou plusieurs heures plus tard.

Le système de coopération asymétrique a également été influencé par plusieurs jeux multijoueurs reposant sur des rôles complémentaires plutôt que sur une symétrie parfaite entre les joueurs. Nous trouvons particulièrement intéressant le fait de donner à chaque joueur une responsabilité totalement différente afin de renforcer la communication et la coordination au sein de la partie.

Sur le plan artistique, la transition vers la 2D nous a amenés à rechercher des inspirations différentes de celles envisagées durant la phase 3D du projet. Nous nous sommes tournés vers plusieurs jeux indépendants utilisant une vue top-down en pixel art, notamment *Project Zomboid* ou *Stardew Valley*, pour la lisibilité de leurs environnements et leur manière d'organiser les espaces de jeu. Bien que nos objectifs de gameplay soient très différents, ces références nous ont aidé à concevoir une carte claire, structurée et facilement compréhensible pour le joueur.

Enfin, certaines inspirations proviennent également directement des contraintes techniques rencontrées durant le projet. En observant différents projets indépendants réalisés avec des outils relativement accessibles, nous avons progressivement compris qu'un style plus simple visuellement pouvait parfois permettre de développer des mécaniques beaucoup plus riches et plus cohérentes sur le plan du gameplay. Cette réflexion a joué un rôle important dans notre décision finale de reconstruire le projet en 2D plutôt que de poursuivre une architecture 3D devenue trop complexe à maintenir dans le temps imparti.

2.5) Les ambitions initiales du projet

Dès les premières semaines de conception, notre ambition pour *Deadly Inspection* était particulièrement importante. Nous ne souhaitons pas simplement réaliser un jeu fonctionnel répondant aux contraintes minimales imposées par le projet académique, mais développer une



expérience coopérative immersive capable de proposer une véritable identité propre. Cette volonté d'aller au-delà d'un simple prototype étudiant a fortement influencé nos premiers choix techniques et artistiques.

Au lancement du projet, nous étions convaincus que la 3D représentait la meilleure solution pour donner vie à notre univers post-apocalyptique. L'objectif était de créer un environnement immersif dans lequel les joueurs pourraient se déplacer librement, observer les survivants sous différents angles et ressentir une véritable présence au sein du refuge. Nous voulions que le joueur ait l'impression d'évoluer dans une base encore vivante malgré l'effondrement du monde extérieur.

Cette ambition se traduisait également dans les mécaniques de gameplay envisagées au départ. Nous souhaitions intégrer des systèmes relativement avancés pour un projet étudiant, notamment un environnement dynamique, des interactions multiples avec le décor, des comportements de personnages plus réalistes ainsi qu'une gestion multijoueur complète permettant à plusieurs joueurs de partager simultanément le même espace de jeu.

Le choix initial du moteur Panda3D s'inscrivait directement dans cette logique. Ce moteur nous semblait suffisamment flexible pour permettre le développement d'un jeu en 3D tout en restant compatible avec le langage Python utilisé durant notre formation. Il offrait également plusieurs fonctionnalités intéressantes pour la gestion du rendu graphique, de la caméra et des déplacements dans un environnement tridimensionnel.

L'une de nos ambitions principales concernait notamment la création de la carte du jeu. Nous souhaitions développer un environnement généré de manière procédurale afin d'éviter une sensation de répétition et de rendre le monde du jeu plus vivant. Pour cela, plusieurs recherches ont été réalisées autour du bruit de Perlin et des techniques de génération procédurale utilisées dans certains jeux vidéo modernes. L'objectif était de produire un terrain dynamique capable de générer automatiquement différents reliefs et environnements.

Le multijoueur représentait également une composante centrale de nos ambitions initiales. Dès le début du projet, nous savions que la communication réseau constituerait l'un des plus grands défis techniques à relever. Nous souhaitions permettre à plusieurs joueurs de se connecter à la même partie tout en conservant une synchronisation fluide des déplacements, des interactions et des événements du jeu. Cette ambition impliquait la mise en place d'une architecture client/serveur ainsi que le développement d'un système de communication réseau relativement complexe pour notre niveau d'expérience.

Parallèlement aux aspects techniques, nous avons également accordé une grande importance à l'immersion générale du projet. Nous voulions créer une atmosphère sombre et oppressante capable de transmettre au joueur une sensation permanente de danger et de fragilité. Cette ambition a concerné autant les décors que les effets sonores, les éclairages ou encore le comportement des survivants rencontrés dans le jeu.



L'objectif n'était donc pas uniquement de produire un jeu "jouable", mais de construire un univers cohérent dans lequel chaque élément contribue à renforcer la tension psychologique ressentie par les joueurs. Cette volonté explique notamment pourquoi nous avons accordé autant d'importance au système de coopération asymétrique entre Bob et Eliott, qui représentait selon nous l'élément le plus original et le plus intéressant du projet.

Cependant, au fil des premières phases de développement, nous avons progressivement pris conscience de l'ampleur réelle des défis techniques liés à ces ambitions. De nombreux systèmes que nous pensions relativement accessibles se sont révélés beaucoup plus complexes à intégrer que prévu, notamment lorsqu'il a fallu faire communiquer plusieurs composants du projet entre eux.

Cette phase initiale du projet a donc été marquée par un mélange d'enthousiasme, d'expérimentation et de découverte. Même si certaines ambitions se sont révélées trop importantes pour être totalement réalisables dans le temps imparti, elles ont joué un rôle essentiel dans la construction du projet et dans notre progression technique tout au long de l'année.

2.6) Les ambitions initiales du projet

Avant même le début du développement concret du jeu, une phase importante de réflexion a été menée afin de définir les grandes orientations techniques et organisationnelles du projet. Cette étape était essentielle, car *Deadly Inspection* représentait un projet beaucoup plus ambitieux et plus long que les exercices réalisés habituellement durant notre formation.

L'un des premiers enjeux concernait la répartition des tâches au sein du groupe. Le projet regroupait plusieurs domaines très différents : développement gameplay, programmation réseau, génération de la carte, interface utilisateur, intégration graphique, gestion sonore et organisation générale du projet. Il était donc nécessaire de structurer rapidement le travail afin d'éviter une désorganisation progressive durant les mois de développement.

Nous avons ainsi choisi de répartir les responsabilités selon les affinités et les compétences de chacun. Certains membres se sont davantage orientés vers les problématiques techniques liées au moteur de jeu et au multijoueur, tandis que d'autres se sont concentrés sur l'interface utilisateur, l'aspect visuel du projet ou encore la gestion globale de l'avancement.

Très rapidement, nous avons également compris que la communication entre les membres du groupe serait un élément déterminant pour assurer la cohérence du projet. Contrairement à des projets individuels plus simples, le développement d'un jeu multijoueur nécessite une forte coordination entre les différentes parties du code. Une modification réalisée sur un système pouvait rapidement impacter le travail des autres membres de l'équipe.



Cette contrainte est devenue particulièrement visible durant le développement de l'architecture 3D. Plusieurs systèmes étaient développés en parallèle : génération procédurale du terrain, gestion des personnages, menu principal, communication réseau ou encore gestion des déplacements. Même si chacun de ces éléments fonctionnait individuellement, leur intégration dans une version unique du jeu s'est révélée beaucoup plus complexe que prévu.

Afin de structurer davantage le développement, nous avons également commencé à utiliser différents outils de gestion de projet et de suivi d'avancement. Des méthodes comme SMART, MOSCOW ou encore les diagrammes de Gantt nous ont permis d'identifier les fonctionnalités prioritaires et de mieux organiser les différentes étapes du développement.

Le versionnage du projet représentait également un enjeu important. Travaillant simultanément sur les mêmes fichiers, nous avons dû apprendre à utiliser Git afin de partager le code, fusionner les modifications et éviter les pertes de données. Cette découverte du travail collaboratif sur un même projet logiciel a constitué une expérience particulièrement formatrice, notamment lors des premiers conflits de merge ou des problèmes d'intégration rencontrés entre différentes branches du projet.

Enfin, cette phase de réflexion nous a progressivement permis de mieux comprendre les réalités concrètes du développement des jeux vidéos. Au-delà des idées initiales et des ambitions créatives, nous avons découvert l'importance de l'organisation, de la gestion du temps et des choix techniques dans la réussite d'un projet informatique de longue durée.

3) Première phase de développement : l'architecture 3D

3.1) Choix de l'architecture technique initiale

Après avoir défini les bases du concept de *Deadly Inspection*, l'équipe a rapidement dû prendre plusieurs décisions techniques majeures concernant l'architecture globale du projet. Cette phase était particulièrement importante, car les choix effectués à ce moment allaient influencer directement l'ensemble du développement futur du jeu.

Notre première grande décision concernait le choix du moteur de jeu. Plusieurs solutions ont été envisagées durant les premières semaines du projet. Nous avons étudié différents frameworks compatibles avec Python ainsi que plusieurs moteurs orientés développement 2D et 3D. Cependant, notre ambition initiale reposait clairement sur la création d'un environnement tridimensionnel immersif, ce qui nous a naturellement orientés vers un moteur capable de gérer ce type d'architecture.

Le moteur Panda3D a finalement été retenu pour plusieurs raisons. Tout d'abord, il présentait l'avantage d'être directement compatible avec Python, langage principal utilisé durant notre



formation à l'EPITA. Cette compatibilité représentait un point important, car elle nous permettait de concentrer nos efforts sur la logique du projet plutôt que sur l'apprentissage complet d'un nouveau langage de programmation.

Panda3D offrait également plusieurs fonctionnalités intéressantes dans le cadre de notre projet. Le moteur proposait une gestion relativement accessible du rendu 3D, des caméras, des lumières, des collisions et des déplacements dans un environnement tridimensionnel. Il permettait également d'accéder à des fonctionnalités plus avancées liées au rendu graphique et à l'intégration de modèles 3D.

Le choix de la 3D répondait avant tout à une volonté d'immersion. Nous voulions créer un univers dans lequel les joueurs auraient réellement l'impression d'évoluer à l'intérieur d'un refuge post-apocalyptique vivant. La 3D nous semblait particulièrement adaptée pour renforcer cette sensation de présence et permettre une interaction plus naturelle avec l'environnement.

Cette ambition a influencé directement plusieurs mécaniques du jeu. Par exemple, le système d'inspection des survivants devait initialement permettre au joueur d'observer les personnages sous différents angles afin de détecter certains détails physiques liés à l'infection. Nous pensions alors que la 3D permettrait de rendre cette mécanique beaucoup plus immersive et réaliste qu'une approche en deux dimensions.

Parallèlement au choix du moteur, nous avons également commencé à réfléchir à l'architecture réseau du projet. Étant donné que *Deadly Inspection* reposait fortement sur la coopération entre plusieurs joueurs, le multijoueur ne pouvait pas être considéré comme une simple fonctionnalité secondaire ajoutée en fin de développement. Au contraire, l'ensemble du projet devait être pensé dès le départ autour de cette contrainte.

Cette réflexion nous a conduits à envisager une architecture client/serveur relativement classique. L'idée était de créer un serveur capable de centraliser les informations importantes de la partie, comme les positions des joueurs, les interactions et les événements, puis de les redistribuer aux différents clients connectés. Même si cette architecture semblait relativement simple en théorie, elle représentait un défi important pour une équipe découvrant encore les bases de la programmation réseau.

Nous avons également commencé à étudier différentes problématiques liées à la synchronisation multijoueur. Très rapidement, nous avons compris qu'un jeu multijoueur temps réel nécessite une organisation du code très différente d'un jeu entièrement local. Chaque déplacement, chaque interaction et chaque événement doit être synchronisé correctement entre les joueurs afin d'éviter des incohérences dans l'état du jeu.

En parallèle, plusieurs recherches ont été menées autour de la génération procédurale du terrain. Notre objectif était de créer une carte capable de générer automatiquement certaines



formes de relief afin d'éviter une sensation de répétition dans l'environnement. Pour cela, nous nous sommes intéressés à différentes techniques utilisées dans les jeux vidéo modernes, notamment l'utilisation du bruit de Perlin pour générer des terrains de manière procédurale.

Cette phase de recherche technique a représenté une étape particulièrement importante dans le projet. Même si une partie de ces systèmes sera finalement abandonnée lors de la transition vers la 2D, elle nous a permis de découvrir de nombreuses notions avancées liées au développement de jeux vidéo, notamment la gestion du rendu 3D, l'architecture réseau, les systèmes procéduraux et l'organisation d'un projet logiciel complexe.

Enfin, cette première phase de développement était également marquée par une forte motivation au sein du groupe. À ce stade du projet, l'équipe était encore portée par une vision très ambitieuse du jeu et par la volonté de construire un univers immersif techniquement avancé. Même si certaines difficultés n'étaient pas encore visibles, cette période a joué un rôle fondamental dans la construction des bases du projet et dans notre progression technique individuelle.

3.2) Premiers prototypes et expérimentations techniques

Une fois les grandes orientations techniques définies, le développement du projet a progressivement commencé à travers la création de plusieurs prototypes expérimentaux. L'objectif de cette phase n'était pas encore de produire un jeu complet, mais plutôt de tester individuellement les différents systèmes techniques nécessaires au fonctionnement futur de *Deadly Inspection*.

Cette approche par prototypes nous permettait d'explorer plusieurs problématiques complexes sans avoir immédiatement à gérer l'intégration complète de tous les systèmes simultanément. Chaque membre du groupe pouvait ainsi se concentrer sur une partie spécifique du projet afin de valider certaines idées ou certains choix techniques avant leur intégration dans une version globale du jeu.

L'un des premiers prototypes développés concernait la génération du terrain en 3D. Plusieurs expérimentations ont été réalisées autour de la génération procédurale à l'aide du bruit de Perlin. L'objectif était de créer automatiquement des reliefs capables de produire une carte plus dynamique et moins répétitive qu'un environnement entièrement construit manuellement.

Cette phase de recherche nous a amenés à travailler directement avec des systèmes relativement avancés de génération de vertices et de rendu GPU. Même si les résultats restaient encore très expérimentaux, cette étape nous a permis de mieux comprendre certaines



problématiques liées aux performances graphiques et à l'optimisation du rendu dans un environnement 3D.

En parallèle, d'autres prototypes ont été développés autour de la gestion des déplacements et de la caméra. Plusieurs systèmes de contrôle ont été testés afin de trouver une solution offrant à la fois une bonne fluidité de déplacement et une lisibilité correcte de l'environnement. Nous avons notamment travaillé sur la gestion des coordonnées, des collisions et du comportement de la caméra dans un espace tridimensionnel.

La communication réseau représentait également une priorité importante durant cette phase. Plusieurs expérimentations ont été menées autour des sockets afin de permettre à plusieurs joueurs de communiquer à travers un serveur centralisé. Ces premiers tests consistaient principalement à envoyer des informations simples, comme les positions des joueurs, les mouvements ou les états de connexion, afin de vérifier la stabilité des échanges réseau.

Même si ces premiers prototypes restaient relativement limités, ils ont rapidement mis en évidence certaines difficultés importantes liées au multijoueur. La synchronisation des déplacements entre plusieurs clients se révélait plus complexe que prévu, notamment lorsque plusieurs joueurs devaient interagir simultanément avec l'environnement.

En parallèle des systèmes techniques principaux, plusieurs expérimentations ont également été réalisées autour de l'interface utilisateur. Des premiers menus ont été créés afin de tester l'affichage des éléments visuels et l'intégration des interactions utilisateur dans Panda3D. Cette partie était relativement plus accessible que le reste du projet, mais elle nécessitait malgré tout un travail important afin d'assurer une cohérence visuelle avec l'univers du jeu.

Cette phase de prototypage avait également un objectif organisationnel important : identifier rapidement les systèmes réellement réalisables dans le temps imparti. À ce stade du projet, certaines idées initiales se révélaient déjà beaucoup plus complexes que prévu, tandis que d'autres semblaient au contraire plus accessibles une fois expérimentées concrètement.

Enfin, ces premiers prototypes ont joué un rôle essentiel dans notre apprentissage global du développement de jeux vidéo. Même si une partie importante de ce travail ne sera pas conservée dans la version finale du projet, cette phase nous a permis d'acquérir de nombreuses compétences techniques qui seront réutilisées par la suite, notamment lors de la reconstruction complète du jeu en 2D.

3.3) Développement du système multijoueur et communication réseau

Le système multijoueur représentait l'un des éléments les plus importants mais également les plus complexes du projet *Deadly Inspection*. Contrairement à un jeu entièrement local, notre projet reposait directement sur l'interaction permanente entre plusieurs joueurs occupant des



rôles complémentaires. Le multijoueur n'était donc pas une fonctionnalité secondaire pouvant être ajoutée en fin de développement, mais une composante centrale qui devait être prise en compte dès les premières étapes de conception.

Très rapidement, nous avons compris que le développement d'une architecture réseau introduit de nombreuses contraintes supplémentaires dans l'organisation globale du projet. Chaque action effectuée par un joueur devait être transmise correctement aux autres joueurs connectés afin de maintenir un état cohérent de la partie. Cela concernait notamment les déplacements, les interactions avec l'environnement, les animations ou encore les différents événements déclenchés dans le jeu.

Afin de répondre à ces besoins, nous avons choisi de mettre en place une architecture client/serveur. Dans ce modèle, un serveur central joue le rôle d'intermédiaire entre les différents joueurs. Chaque client envoie régulièrement ses informations au serveur, qui redistribue ensuite les données mises à jour à l'ensemble des joueurs connectés. Cette architecture présentait plusieurs avantages, notamment une meilleure centralisation des données et une synchronisation plus cohérente de l'état du jeu.

Le développement du serveur a constitué l'une des premières grandes difficultés techniques du projet. Une partie importante du travail a consisté à comprendre le fonctionnement des sockets réseau et des échanges de données entre plusieurs machines. Cette problématique représentait un domaine relativement nouveau pour l'ensemble du groupe, ce qui a nécessité une importante phase d'apprentissage à travers des documentations, des exemples de code et différents tutoriels spécialisés.

Les premiers tests multijoueurs reposaient sur des échanges très simples. L'objectif initial était simplement de permettre à deux joueurs de se connecter simultanément au serveur puis d'afficher leurs positions respectives dans le jeu. Même cette première étape s'est révélée plus complexe que prévu, notamment en raison des problèmes de synchronisation et de transmission des données en temps réel.

Au fur et à mesure des expérimentations, plusieurs difficultés sont progressivement apparues. L'une des principales concernait la synchronisation des déplacements des joueurs. Dans certains cas, les mouvements d'un joueur n'étaient pas correctement répercutés chez les autres clients, provoquant des incohérences importantes dans l'affichage du jeu. Ces problèmes devenaient particulièrement visibles lorsque plusieurs actions étaient effectuées simultanément.

La gestion des délais réseau représentait également une difficulté importante. Même sur un réseau local relativement stable, de légers retards pouvaient provoquer des comportements imprévus ou des décalages entre les différents clients connectés. Nous avons alors progressivement compris qu'un jeu multijoueur temps réel nécessite une gestion beaucoup plus rigoureuse des états et des mises à jour qu'un jeu classique entièrement local.



Parallèlement au serveur principal, nous avons également commencé à travailler sur certaines problématiques liées à l'accessibilité du multijoueur. L'objectif était de permettre à plusieurs joueurs situés sur des réseaux différents de rejoindre facilement une même partie sans nécessiter de configuration réseau complexe. C'est dans cette optique qu'un travail de recherche a été mené autour de l'UPnP (Universal Plug and Play).

L'UPnP permet théoriquement d'ouvrir automatiquement certains ports réseau sur la box Internet de l'hôte afin de faciliter les connexions entrantes. Cette fonctionnalité représentait un avantage important pour rendre le multijoueur plus accessible à des joueurs ne possédant pas de connaissances avancées en configuration réseau. Cependant, sa mise en place s'est révélée particulièrement complexe, notamment en raison des différences de configuration entre les réseaux et des difficultés de test dans un environnement étudiant.

Même si ces systèmes restaient encore expérimentaux à ce stade du projet, cette phase de développement nous a permis de mieux comprendre les enjeux réels liés au développement d'un jeu multijoueur. Nous avons notamment découvert que la programmation réseau ne consiste pas simplement à "connecter" plusieurs joueurs, mais nécessite une réflexion complète sur la synchronisation des données, la gestion des états du jeu et la stabilité générale de l'architecture.

Enfin, cette phase a également mis en évidence un problème qui deviendra progressivement central dans la suite du projet : l'intégration des différents systèmes entre eux. Même si plusieurs prototypes fonctionnent individuellement, leur fusion dans une version globale stable du jeu devenait de plus en plus difficile au fur et à mesure de l'avancement du développement.

3.4) Génération environnement 3D

En parallèle du développement du multijoueur, une partie importante du projet concernait la création de l'environnement du jeu et la génération de la carte en trois dimensions. Dès le début du projet, nous voulions éviter une carte entièrement fixe ou trop répétitive. Notre objectif était de concevoir un environnement capable de donner une impression de monde vivant et relativement dynamique malgré les contraintes du projet.

Pour répondre à cette ambition, nous avons commencé à nous intéresser à la génération procédurale. Cette technique, utilisée dans de nombreux jeux vidéo modernes, consiste à générer automatiquement certains éléments du décor à l'aide d'algorithmes plutôt que de construire manuellement l'intégralité de la carte. Cette approche présentait plusieurs avantages, notamment la possibilité de produire des reliefs plus naturels et de réduire le travail de modélisation manuelle.



Nos recherches se sont principalement orientées vers l'utilisation du bruit de Perlin. Cet algorithme permet de générer des variations progressives utilisées pour créer des reliefs, des terrains ou différents types de structures naturelles. L'objectif était d'utiliser ce système afin de produire automatiquement une carte possédant différents niveaux de hauteur et une apparence moins artificielle qu'une simple surface plane.

La mise en place de cette génération procédurale représentait cependant un défi technique important. Le système nécessitait la création dynamique de nombreux vertices ainsi que leur envoi vers le GPU afin d'assurer le rendu du terrain. Cette problématique nous a amenés à découvrir plusieurs notions avancées liées au rendu graphique et à l'optimisation des performances dans un environnement 3D.

Même si les premiers résultats obtenus restaient relativement expérimentaux, cette phase de développement nous a permis d'obtenir un premier environnement jouable dans lequel les joueurs pouvaient se déplacer librement. Cette étape représentait un moment particulièrement motivant dans le projet, car elle donnait enfin une forme concrète à l'univers que nous imaginions depuis les premières phases de conception.

Parallèlement à la génération du terrain, plusieurs expérimentations ont également été réalisées autour des déplacements de la caméra et des contrôles du joueur. Nous souhaitions obtenir une caméra suffisamment fluide pour permettre une bonne lisibilité de l'environnement tout en conservant une sensation d'immersion adaptée à l'univers du jeu.

La gestion des collisions représentait également une problématique importante. Dans un environnement 3D, empêcher correctement les personnages de traverser les objets du décor nécessite une gestion relativement précise des volumes de collision et des interactions physiques. Cette partie du projet s'est révélée plus complexe que prévu, notamment lorsque plusieurs systèmes de déplacement et de caméra étaient utilisés simultanément.

Les décors constituaient eux aussi une difficulté importante. Contrairement à un environnement 2D où les éléments visuels peuvent être placés relativement simplement, la 3D nécessite l'intégration de modèles, de textures, de lumières et parfois d'animations supplémentaires. Chaque nouvel élément ajouté à la scène augmentait progressivement la complexité générale du projet ainsi que les risques d'incompatibilité entre les différents systèmes.

Au fil du développement, nous avons également commencé à observer certaines limites importantes concernant les performances et la stabilité globale de cette architecture 3D. Même si plusieurs prototypes fonctionnaient individuellement, leur intégration dans une version complète du jeu devenait de plus en plus difficile à maintenir. Les problèmes liés aux collisions, à la caméra, au rendu graphique et au multijoueur commençaient progressivement à s'accumuler.



Cependant, malgré ces difficultés, cette phase du projet a joué un rôle extrêmement formateur. Elle nous a permis de découvrir des notions avancées rarement abordées dans des projets étudiants de première année, notamment la génération procédurale, le rendu GPU, la gestion des environnements 3D et certaines problématiques liées à l'optimisation graphique.

Enfin, cette étape a également permis au groupe de prendre progressivement conscience de l'ampleur réelle du projet. Les ambitions initiales restaient techniquement intéressantes, mais leur réalisation complète dans le temps imparti devenait de plus en plus difficile à envisager sans repenser une partie importante de l'architecture globale du jeu.

4) Les limites du prototypes 3D

4.1) Une complexité technique plus importante que prévue

Au cours des premières phases de développement, l'architecture 3D de *Deadly Inspection* nous avait permis de produire plusieurs prototypes techniquement intéressants. Cependant, au fur et à mesure de l'avancement du projet, les limites de cette approche sont progressivement devenues de plus en plus visibles. Ce qui apparaissait au départ comme une architecture ambitieuse et immersive commençait à devenir une source importante de difficultés techniques et organisationnelles.

L'un des principaux problèmes rencontrés concernait la multiplication des systèmes à gérer simultanément. Le projet reposait déjà sur plusieurs éléments complexes : génération procédurale, communication réseau, gestion des déplacements, rendu graphique, collisions, caméra, interface utilisateur et synchronisation multijoueur. Pris individuellement, certains de ces systèmes fonctionnaient relativement correctement. En revanche, leur intégration dans une version unique stable du jeu devenait extrêmement difficile.

La 3D ajoutait une couche supplémentaire de complexité à pratiquement tous les aspects du développement. Chaque nouveau système nécessitait davantage de calculs, davantage de gestion graphique et davantage de coordination entre les différentes parties du projet. Par exemple, une simple interaction avec l'environnement nécessitait souvent la gestion simultanée des collisions, des animations, des coordonnées 3D et parfois de la synchronisation réseau si plusieurs joueurs étaient concernés.

Les animations représentaient également une difficulté particulièrement importante. L'intégration de personnages en 3D nécessitait la gestion des articulations, des déplacements, des orientations ainsi que des transitions entre différentes animations. Cette partie du développement s'est révélée beaucoup plus complexe que prévu pour une équipe découvrant encore les bases du développement de jeux vidéo.



Le système de caméra posait lui aussi de nombreux problèmes. Nous cherchions à obtenir un compromis entre lisibilité du gameplay et immersion dans l'environnement. Cependant, la gestion de la caméra dans un espace tridimensionnel entraînait régulièrement des comportements imprévus : angles difficiles à contrôler, problèmes de visibilité ou collisions avec certains éléments du décor.

La génération procédurale du terrain, bien qu'intéressante techniquement, augmentait également fortement la difficulté globale du projet. Le terrain devait être généré dynamiquement, affiché correctement par le moteur graphique puis rester compatible avec les déplacements, les collisions et les autres systèmes du jeu. Cette accumulation de contraintes techniques rendait progressivement le projet beaucoup plus difficile à maintenir.

Le multijoueur a amplifié davantage ces difficultés. Chaque élément du jeu devait désormais fonctionner non seulement localement, mais également être synchronisé correctement entre plusieurs clients connectés. Une erreur mineure dans la gestion d'un déplacement ou d'une interaction pouvait rapidement provoquer des incohérences importantes entre les joueurs.

Très rapidement, nous avons commencé à constater que le temps passé à corriger des bugs techniques devenait parfois plus important que le temps consacré à développer de nouvelles fonctionnalités. Certains problèmes semblaient résolus individuellement mais réapparaissaient dès qu'un autre système était modifié ou ajouté au projet.

Cette situation a progressivement entraîné une forme de fragmentation du développement. Chaque membre du groupe avançait sur sa propre partie du projet, mais l'intégration globale devenait de plus en plus difficile à réaliser. Plusieurs prototypes fonctionnaient séparément : carte, serveur, déplacements, menu, sans réussir à former un ensemble totalement cohérent et stable.

Cette phase du projet a été particulièrement importante dans notre apprentissage. Elle nous a permis de comprendre qu'un projet techniquement ambitieux ne dépend pas uniquement de la qualité individuelle des systèmes développés, mais surtout de leur capacité à fonctionner ensemble de manière cohérente et maintenable.

Enfin, cette accumulation de difficultés nous a progressivement amenés à remettre en question certains choix réalisés au début du projet. Même si la 3D restait intéressante sur le plan de l'immersion et de l'ambition technique, son coût en temps et en complexité devenait de plus en plus difficile à justifier au regard des contraintes du projet étudiant.

4.2) Les difficultés d'intégration et la fragmentation du projet

Au fur et à mesure de l'avancement du développement, l'un des problèmes majeurs rencontrés par l'équipe concernait l'intégration des différents systèmes du projet dans une



version unique stable et réellement jouable. Même si plusieurs fonctionnalités avaient été développées avec succès individuellement, leur fusion progressive révélait de nombreuses incompatibilités techniques et organisationnelles.

Le projet était progressivement devenu composé de plusieurs “blocs” techniques relativement indépendants : génération procédurale de la carte, système multijoueur, gestion des déplacements, interface utilisateur, caméra, collisions ou encore architecture réseau. Chacun de ces éléments représentait déjà un travail important à lui seul, mais leur communication entre eux restait extrêmement difficile à maintenir.

Cette fragmentation du développement s’expliquait en partie par l’organisation du travail en parallèle. Afin d’avancer plus rapidement, les membres du groupe travaillaient simultanément sur différentes fonctionnalités. Cette méthode présentait certains avantages en termes de productivité, mais elle augmentait également les risques d’incompatibilité lors de l’intégration finale des systèmes.

Par exemple, certains prototypes de déplacements fonctionnaient correctement dans un environnement local simple, mais provoquaient des comportements imprévus lorsqu’ils étaient intégrés au système multijoueur. De la même manière, certaines modifications apportées à la caméra ou au rendu graphique pouvaient perturber le fonctionnement des collisions ou des interactions avec l’environnement.

Le problème devenait encore plus visible lors des tentatives de fusion du projet dans un unique exécutable fonctionnel. Même lorsque plusieurs systèmes semblaient fonctionner correctement individuellement, leur assemblage dans une version globale provoquait régulièrement de nouveaux bugs ou des pertes de stabilité importantes.

Cette difficulté était particulièrement frustrante pour l’équipe, car elle donnait parfois l’impression que le projet avançait techniquement sans réellement se rapprocher d’une version jouable complète. Plusieurs démonstrations pouvaient être réalisées séparément, mais il devenait de plus en plus compliqué de présenter un véritable “jeu” cohérent regroupant l’ensemble des fonctionnalités prévues.

Le travail collaboratif sur Git a également mis en évidence certaines limites dans notre organisation. Les merges entre différentes branches du projet entraînaient parfois des conflits complexes, notamment lorsque plusieurs membres modifiaient simultanément des systèmes liés entre eux. Une modification mineure dans une partie du code pouvait parfois avoir des conséquences imprévues sur d’autres fonctionnalités du projet.

Cette phase nous a permis de prendre conscience de l’importance de l’architecture logicielle dans un projet de développement de longue durée. Nous avons progressivement compris qu’un système techniquement impressionnant mais difficile à intégrer peut devenir un obstacle majeur pour l’avancement global du projet.



Par ailleurs, certaines ambitions initiales devenaient de plus en plus difficiles à maintenir dans le temps imparti. Chaque nouvelle fonctionnalité ajoutée augmentait encore davantage la complexité générale du projet, ce qui ralentissait progressivement le développement global. Au lieu d'obtenir une progression régulière, nous passions parfois plusieurs jours à corriger des problèmes d'intégration ou de compatibilité entre différents systèmes.

Cette situation a progressivement conduit l'équipe à s'interroger sur la viabilité de l'architecture choisie au départ. Même si plusieurs prototypes restaient prometteurs individuellement, le projet dans son ensemble devenait de plus en plus difficile à stabiliser et à faire évoluer correctement.

Enfin, cette phase de fragmentation a joué un rôle déterminant dans la réflexion qui conduira plus tard à la transition complète vers la 2D. Elle nous a notamment permis de comprendre qu'un projet techniquement plus simple mais cohérent et jouable pouvait finalement être beaucoup plus pertinent qu'une architecture très ambitieuse mais difficilement exploitable dans le cadre du temps disponible.

4.3) Contrainte temporelles et remise en question du projet

Au-delà des difficultés purement techniques, l'un des éléments ayant fortement influencé l'évolution du projet concernait la gestion du temps et des contraintes liées au calendrier académique. Comme l'ensemble des groupes de première année à l'EPITA, nous devions développer notre projet en parallèle des cours, des travaux pratiques, des examens et des autres projets imposés durant l'année. Cette contrainte limitait fortement le temps réellement disponible pour travailler sur *Deadly Inspection*.

Au début du projet, cette contrainte nous paraissait relativement gérable. L'équipe était particulièrement motivée et les premiers prototypes donnaient l'impression d'une progression rapide. Cependant, au fil des semaines, l'augmentation progressive de la complexité technique du projet a commencé à ralentir considérablement l'avancement global du développement.

Une partie importante du temps de travail était désormais consacrée à la résolution de problèmes techniques parfois très spécifiques : bugs de synchronisation réseau, erreurs liées au rendu graphique, problèmes de collisions, incompatibilités entre différents systèmes ou encore difficultés liées à l'intégration des fonctionnalités développées séparément. Même lorsqu'un problème semblait résolu, il réapparaissait parfois sous une autre forme après l'ajout d'une nouvelle fonctionnalité.

Cette situation a progressivement modifié notre manière de travailler. Au lieu d'avancer régulièrement vers la création de nouvelles mécaniques de gameplay, nous passions parfois



plusieurs séances entières à tenter de stabiliser des systèmes déjà existants. Cela créait une forme de frustration au sein du groupe, car l'impression de progression devenait beaucoup moins visible malgré le temps investi dans le projet.

Par ailleurs, certaines ambitions initiales du projet devenaient de plus en plus difficiles à atteindre dans les délais disponibles. Plusieurs mécaniques prévues au départ nécessitaient encore un important travail de développement alors que les bases du jeu elles-mêmes restaient encore instables. Cette accumulation de tâches provoquait progressivement un déséquilibre entre l'ampleur du projet imaginé et les ressources réellement disponibles pour le réaliser.

Les soutenances intermédiaires ont également joué un rôle important dans cette prise de conscience. Même si nous pouvions présenter plusieurs systèmes fonctionnels séparément, il devenait évident que le projet manquait encore d'une véritable cohérence globale. Le jeu existait davantage sous forme de prototypes techniques que comme une expérience réellement jouable et complète.

Cette période a marqué un tournant important dans notre réflexion sur le projet. Nous avons progressivement compris qu'il devenait nécessaire de hiérarchiser plus strictement les objectifs et de distinguer les fonctionnalités réellement essentielles de celles qui relevaient davantage de l'ambition technique ou visuelle.

L'équipe a alors commencé à remettre en question certains choix réalisés au lancement du projet, notamment l'utilisation de la 3D. Même si cette architecture restait intéressante sur le plan technique, elle mobilisait une quantité très importante de temps pour maintenir des systèmes relativement basiques. Certaines fonctionnalités qui semblaient simples en théorie demandaient finalement plusieurs semaines de développement et de corrections avant d'obtenir un résultat relativement stable.

Cette réflexion nous a amenés à considérer une idée qui semblait initialement difficile à accepter : la possibilité de reconstruire entièrement le projet dans une architecture plus simple et plus adaptée aux contraintes réelles du développement. Cette décision représentait un risque important, car elle impliquait potentiellement l'abandon d'une grande partie du travail déjà réalisé depuis plusieurs mois.

Cependant, au fil des discussions, cette solution apparaissait progressivement comme la seule manière réaliste d'aboutir à une version réellement jouable et cohérente de *Deadly Inspection* avant la soutenance finale. Il ne s'agissait plus uniquement de préserver l'ambition technique initiale du projet, mais surtout de garantir la création d'une véritable expérience de jeu fonctionnelle.

Enfin, cette phase de remise en question a représenté un apprentissage particulièrement important pour l'ensemble du groupe. Elle nous a permis de comprendre qu'en



développement logiciel, savoir adapter ses ambitions aux contraintes réelles du projet constitue parfois une compétence aussi importante que les capacités purement techniques.

4.4) Abandon de la 3D et transition vers la 2D

La décision d'abandonner entièrement l'architecture 3D du projet a constitué le tournant le plus important du développement de *Deadly Inspection*. Cette transition ne s'est pas faite de manière immédiate, mais résulte d'une réflexion progressive menée par l'ensemble du groupe après plusieurs mois de développement et de difficultés techniques accumulées.

Au départ, l'idée même d'abandonner la 3D semblait difficilement envisageable. Une grande partie du travail réalisé depuis le début du projet reposait sur cette architecture : génération procédurale du terrain, prototypes multijoueurs, gestion des déplacements, caméra ou encore différents systèmes graphiques. Repartir sur une nouvelle base signifiait accepter qu'une partie importante de ces développements ne serait finalement pas conservée dans la version finale du jeu.

Cependant, plus le projet avançait, plus les limites de cette architecture devenaient évidentes. Malgré les efforts fournis par l'équipe, nous avions de plus en plus de difficultés à transformer nos différents prototypes en un véritable jeu stable et cohérent. Le projet fonctionnait par fragments : certains systèmes étaient techniquement intéressants, mais leur intégration globale restait extrêmement fragile.

Cette situation posait également un problème important concernant l'expérience de jeu elle-même. Même si plusieurs démonstrations techniques pouvaient être présentées séparément, nous ne disposions toujours pas d'une version réellement jouable capable de mettre en valeur le concept principal de *Deadly Inspection*. Or, notre objectif n'était pas uniquement de produire une démonstration technique, mais bien de créer une expérience coopérative fonctionnelle et immersive.

La réflexion autour du passage à la 2D s'est alors progressivement imposée comme une solution réaliste pour recentrer le projet sur ses mécaniques principales plutôt que sur la complexité de son architecture graphique. Nous avons compris qu'une partie importante de notre temps était consacrée à résoudre des problèmes liés directement à la 3D, sans que cela améliore réellement le cœur du gameplay.

Le passage à la 2D présentait plusieurs avantages immédiats. Tout d'abord, il permettait de simplifier énormément la gestion des déplacements, des collisions et des interactions avec l'environnement. Là où la 3D nécessitait une gestion complexe des volumes, des angles de caméra et des modèles, une architecture 2D permettait de se concentrer beaucoup plus directement sur le gameplay lui-même.



Ce changement nous offrait également la possibilité de reconstruire plus rapidement une base de jeu stable et cohérente. Certaines fonctionnalités qui demandaient plusieurs semaines de travail dans l'environnement 3D devenaient beaucoup plus accessibles dans une architecture 2D basée sur Pygame.

Le choix de Pygame s'est rapidement imposé pour accompagner cette transition. Cette bibliothèque, spécialisée dans le développement de jeux 2D en Python, proposait une approche beaucoup plus légère et plus adaptée à nos besoins. Elle permettait également de rester dans un environnement de développement cohérent avec les compétences acquises durant notre formation.

La transition vers la 2D ne signifiait cependant pas l'abandon du concept initial du projet. Au contraire, notre objectif était de préserver l'essence même de *Deadly Inspection* — coopération asymétrique, gestion du refuge, analyse des survivants, crises internes — tout en reconstruisant l'architecture technique du jeu dans un environnement plus réaliste à développer dans le temps imparti.

Cette décision a marqué un changement important dans notre manière d'aborder le projet. Là où les premières phases de développement étaient principalement guidées par l'ambition technique, cette nouvelle approche cherchait avant tout à privilégier la cohérence globale, la jouabilité et la stabilité de l'expérience proposée au joueur.

Avec le recul, cette transition représente probablement la décision la plus importante prise durant toute la durée du projet. Même si elle impliquait de repartir presque entièrement de zéro sur le plan technique, elle a permis de transformer progressivement *Deadly Inspection* en un véritable jeu jouable plutôt qu'en une succession de prototypes difficiles à intégrer ensemble.

5) Reconstruction complète du projet en 2D

5.1) Reconstruction technique nécessaire

La transition vers la 2D a marqué le début d'une nouvelle phase de développement pour *Deadly Inspection*. Contrairement à une simple modification graphique, ce changement représentait en réalité une reconstruction presque complète du projet sur le plan technique. Une grande partie des systèmes développés durant la phase 3D ne pouvait pas être réutilisée directement dans la nouvelle architecture, ce qui a obligé l'équipe à repenser entièrement l'organisation du jeu.

Même si cette décision impliquait l'abandon d'une partie importante du travail déjà réalisé, elle a rapidement produit des effets positifs sur l'avancement global du projet. Là où le



développement en 3D ralentissait progressivement à cause de la complexité technique et des difficultés d'intégration, l'environnement 2D permettait de reconstruire beaucoup plus rapidement des systèmes stables et cohérents.

Cette phase de reconstruction a également été l'occasion de réévaluer plusieurs choix réalisés durant les premières étapes du projet. Nous avons cherché à simplifier certains systèmes devenus inutilement complexes dans la version 3D afin de recentrer le développement sur les mécaniques réellement importantes pour l'expérience de jeu.

Le choix de Pygame comme nouvelle base technique a joué un rôle central dans cette reconstruction. Contrairement à Panda3D, principalement orienté vers la 3D, Pygame proposait une architecture beaucoup plus légère et mieux adaptée au type de jeu que nous souhaitions désormais développer. Cette bibliothèque permettait de gérer relativement simplement l'affichage des sprites, les déplacements, les collisions, les interactions utilisateur ainsi que les systèmes sonores.

L'un des premiers avantages observés concernait la rapidité de développement. Certaines fonctionnalités qui nécessitaient auparavant plusieurs jours, voire plusieurs semaines de travail dans l'environnement 3D, pouvaient désormais être implémentées en quelques heures seulement. Cette différence de productivité a profondément modifié notre manière d'aborder le projet.

Le passage à la 2D a également permis de simplifier considérablement la gestion des personnages et des animations. Dans la version 3D, l'intégration des modèles nécessitait la gestion des articulations, des orientations et parfois des animations complexes. Avec Pygame, les personnages reposaient désormais sur des sprites 2D beaucoup plus simples à manipuler et à intégrer dans le moteur du jeu.

Cette simplification ne signifiait cependant pas une diminution de l'ambition globale du projet. Au contraire, en réduisant la complexité liée au rendu graphique, nous pouvions désormais consacrer davantage de temps au développement des mécaniques principales du gameplay : coopération asymétrique, analyse des survivants, gestion du refuge et interactions entre les joueurs.

Le changement d'architecture nous a également permis d'améliorer progressivement la stabilité globale du projet. Les systèmes devenaient plus faciles à tester, plus simples à corriger et surtout beaucoup plus simples à intégrer ensemble. Là où la version 3D souffrait d'une importante fragmentation, la reconstruction 2D nous a permis de repartir sur une base beaucoup plus cohérente.

Cette période a également marqué une évolution importante dans notre approche du développement. Nous avons progressivement appris à privilégier la cohérence et la jouabilité plutôt que la complexité technique pure. Cette réflexion a joué un rôle essentiel dans



l'amélioration globale du projet et dans notre compréhension des contraintes réelles du développement de jeux vidéo.

Enfin, cette reconstruction a permis de redonner une dynamique positive au projet. Après plusieurs mois marqués par des difficultés techniques importantes, l'équipe retrouvait progressivement une impression de progression concrète. Chaque nouvelle fonctionnalité ajoutée contribuait désormais directement à la création d'un véritable jeu jouable et non plus uniquement à l'expérimentation technique de systèmes isolés.

5.2) Nouvelle direction artistique et identité visuelle

Le passage à la 2D ne concernait pas uniquement les aspects techniques du projet. Cette transition a également entraîné une refonte complète de la direction artistique et de l'identité visuelle de *Deadly Inspection*. L'environnement tridimensionnel initial reposait sur une recherche d'immersion réaliste, tandis que la nouvelle architecture nécessitait une approche graphique différente, mieux adaptée au gameplay et aux contraintes techniques du projet.

Après plusieurs réflexions, nous avons choisi d'adopter une vue "top-down" en pixel art. Ce style graphique présente plusieurs avantages dans le cadre d'un projet étudiant. Tout d'abord, il permet de créer rapidement des environnements lisibles et cohérents sans nécessiter des compétences avancées en modélisation 3D. Ensuite, il reste particulièrement efficace pour les jeux reposant sur la gestion, l'exploration et les interactions avec l'environnement.

Cette direction artistique a été fortement influencée par plusieurs jeux indépendants utilisant des environnements 2D similaires. Des titres comme *Project Zomboid* nous ont notamment inspirés dans la manière de représenter un univers post-apocalyptique vu du dessus, avec une forte importance accordée à la lisibilité de l'environnement et aux interactions entre les personnages et le décor.

Le pixel art présentait également un avantage important concernant la cohérence visuelle du projet. Contrairement à la 3D, où les différences de qualité entre les modèles, les textures et les animations pouvaient rapidement devenir visibles, le style pixel art permettait de construire un univers plus homogène à partir de ressources graphiques relativement simples.

L'ambiance générale du jeu restait cependant fidèle à notre vision initiale. Nous souhaitons toujours créer une atmosphère sombre et oppressante, reflétant un monde détruit par une infection ayant provoqué l'effondrement de la société. Les décors, les couleurs et les différents environnements ont donc été pensés pour transmettre cette sensation permanente de fragilité et de tension.

Le refuge constitue notamment l'élément central de cette direction artistique. Même en 2D, nous voulions que la base donne l'impression d'un espace encore vivant malgré



l'environnement hostile extérieur. Les différentes zones, comme le laboratoire, le poste de contrôle, la boutique, l'infirmerie ou les espaces de repos, ont été organisées de manière à rendre le refuge crédible et facilement compréhensible pour le joueur.

La lisibilité représentait d'ailleurs un objectif particulièrement important dans cette nouvelle direction artistique. Contrairement à la version 3D où la caméra pouvait parfois compliquer la visibilité, l'environnement top-down permettait de mieux organiser les informations à l'écran et de rendre les interactions beaucoup plus intuitives.

Nous avons également commencé à accorder davantage d'importance à l'interface utilisateur et à l'ambiance sonore du projet. Les menus, les boutons et les différentes interfaces ont progressivement été retravaillés afin de mieux correspondre à l'identité visuelle du jeu. L'ajout de musiques et de sons d'ambiance a également permis de renforcer l'immersion malgré la simplification graphique liée au passage à la 2D.

Enfin, cette nouvelle direction artistique a profondément modifié notre perception du projet lui-même. Là où la version 3D semblait parfois très ambitieuse mais difficilement maîtrisable, la version 2D donnait progressivement naissance à un univers beaucoup plus cohérent, lisible et réellement jouable. Cette évolution a joué un rôle essentiel dans la reconstruction globale de *Deadly Inspection*.

5.3) Conception de la carte et utilisation de Tiled

L'un des premiers objectifs de cette nouvelle phase de développement consistait à reconstruire entièrement l'environnement du jeu dans une architecture 2D cohérente et facilement exploitable. Contrairement à la phase 3D, où la génération procédurale occupait une place importante, nous avons choisi d'adopter une approche beaucoup plus structurée et maîtrisée pour la création de la carte.

Pour cela, nous avons décidé d'utiliser le logiciel Tiled, un éditeur de niveaux largement utilisé dans le développement de jeux vidéo en 2D. Cet outil permet de construire des cartes à partir de "tiles", c'est-à-dire de petites images assemblées afin de former un environnement complet. Ce choix représentait un avantage considérable dans le cadre du projet, car il nous permettait de concevoir visuellement les différentes zones du jeu tout en gardant une organisation claire et modifiable facilement.

L'utilisation de Tiled a profondément simplifié le processus de création de la map. Là où la version 3D nécessitait une gestion complexe des modèles, des textures et du terrain procédural, la 2D nous permettait désormais de construire rapidement un environnement fonctionnel à partir d'éléments graphiques relativement simples.



Nous avons organisé la carte autour de plusieurs zones principales correspondant directement aux mécaniques du gameplay. Le refuge est ainsi divisé en différents espaces ayant chacun une fonction précise dans l'expérience de jeu. Le poste de contrôle représente la zone

principale de Bob, où les survivants viennent se présenter afin d'être inspectés avant leur admission dans la base. Cette zone constitue l'un des centres névralgiques du gameplay, puisqu'elle concentre une grande partie des décisions importantes du jeu.

Le laboratoire d'Eliott représente quant à lui un espace davantage orienté vers la gestion et la recherche. Cette zone a été pensée comme un lieu stratégique permettant au joueur de développer des outils, gérer les ressources du refuge et réagir aux différentes crises apparaissant durant la partie.

D'autres zones ont également été intégrées afin de renforcer la crédibilité du refuge : boutique, infirmerie, espaces de repos, tentes de survivants ou encore différentes structures défensives. Même si toutes ces zones ne possèdent pas encore un gameplay totalement développé, leur présence contribue à donner une cohérence globale à l'environnement du jeu.

La conception de la carte ne reposait pas uniquement sur l'aspect visuel. Nous avons également dû réfléchir à la circulation des joueurs dans l'environnement et à la lisibilité générale du gameplay. Chaque zone devait être facilement identifiable afin que les joueurs comprennent rapidement où se rendre et quelles interactions effectuer.

Cette réflexion sur l'organisation spatiale représentait une amélioration importante par rapport à la version 3D. Dans l'environnement tridimensionnel initial, certains problèmes de caméra ou de perspective rendaient parfois les déplacements moins intuitifs. Avec la vue top-down, la lecture de la carte devenait beaucoup plus naturelle et facilitait considérablement la compréhension du jeu.

L'intégration de la carte dans Pygame a également nécessité plusieurs ajustements techniques. Une fois exportée depuis Tiled, la map devait être interprétée correctement par le moteur du jeu afin de gérer l'affichage des différents éléments, les collisions et les interactions avec l'environnement. Cette étape nous a permis de mieux comprendre la manière dont les données visuelles peuvent être reliées directement à la logique du gameplay.

Les collisions représentaient notamment une partie importante de ce travail. Certains objets de la carte, comme les bâtiments, les barrières ou différents éléments du décor, devaient empêcher les déplacements du joueur afin de rendre l'environnement crédible. Nous avons donc dû définir différentes zones de collision et programmer leur comportement dans Pygame.

Au-delà de son intérêt technique, cette phase de conception de la carte a également marqué un moment important dans le développement du projet. Pour la première fois, nous avons

réellement l'impression de construire un environnement de jeu cohérent et directement exploitable dans une version jouable de *Deadly Inspection*.

Enfin, l'utilisation de Tiled a joué un rôle essentiel dans l'accélération globale du développement. Grâce à cet outil, la création de nouveaux environnements, la modification de certaines zones ou l'ajout d'éléments interactifs devenaient beaucoup plus rapides et beaucoup plus simples à gérer que dans l'ancienne architecture 3D.

5.4) Développement des interactions et des systèmes de gameplay

Une fois la nouvelle architecture 2D stabilisée et la carte du jeu mise en place, le développement s'est progressivement concentré sur les différentes interactions composant le gameplay de *Deadly Inspection*. Cette étape représentait un changement important dans le projet, car nous passions enfin d'une logique principalement technique à une véritable construction de l'expérience de jeu.

L'un des premiers objectifs consistait à rendre l'environnement interactif afin que les joueurs puissent réellement agir sur le refuge et interagir avec les différents éléments présents sur la carte. Plusieurs systèmes ont alors commencé à être développés autour des zones importantes du jeu, notamment le poste de contrôle, la boutique et le laboratoire.

Le poste de contrôle représente la principale zone d'interaction de Bob. Lorsqu'un survivant se présente à l'entrée du refuge, le joueur doit pouvoir examiner certaines informations, analyser différents indices puis prendre une décision concernant son admission. Cette mécanique constitue le cœur du gameplay de *Deadly Inspection*, car elle génère directement les différentes conséquences affectant la survie du refuge.

Nous avons progressivement commencé à développer un système permettant de générer différents profils de survivants. Chaque personnage possède certaines caractéristiques spécifiques ainsi que des indices plus ou moins visibles pouvant laisser penser à une infection. L'objectif était de créer une véritable tension psychologique chez le joueur, qui doit constamment décider s'il peut faire confiance aux individus se présentant à la base.

Le laboratoire représente quant à lui l'une des principales interactions d'Eliott. Même si cette partie du gameplay reste encore en cours de développement, l'objectif est de permettre au joueur de gérer les ressources scientifiques du refuge, développer de nouveaux outils et améliorer progressivement les capacités de survie de la communauté.

Cette séparation des interactions entre les deux rôles permet de renforcer fortement l'asymétrie coopérative du projet. Bob agit principalement sur les menaces extérieures tandis



qu'Eliott gère les conséquences internes et l'évolution du refuge. Cette organisation crée une dépendance permanente entre les deux joueurs et renforce la communication nécessaire au bon déroulement de la partie.

La boutique constitue également un élément important du gameplay. Elle permet d'introduire un système économique relativement simple basé sur les ressources obtenues grâce aux survivants acceptés dans la base. Cette mécanique ajoute une dimension stratégique supplémentaire, car chaque décision prise par les joueurs influence directement les capacités futures du refuge.

Parallèlement à ces systèmes principaux, plusieurs interactions secondaires ont également été développées afin de rendre l'environnement plus vivant. Certaines zones déclenchent des événements spécifiques, des menus d'interaction ou des changements d'état du jeu lorsque le joueur s'en approche.

L'interface utilisateur a également beaucoup évolué durant cette phase. Contrairement à la version 3D où les menus restaient relativement expérimentaux, la reconstruction 2D nous a permis de développer des interfaces beaucoup plus cohérentes avec le gameplay. Des fenêtres d'interaction, des boutons, des éléments de texte et différents systèmes d'information ont progressivement été intégrés afin d'améliorer la lisibilité du jeu.

Le système sonore a lui aussi commencé à prendre une place plus importante dans l'expérience globale. Des musiques d'ambiance ainsi que différents effets sonores ont été ajoutés afin de renforcer l'immersion et de rendre l'environnement plus vivant malgré la simplicité graphique relative de la 2D.

Cette phase du développement représente probablement le moment où *Deadly Inspection* a commencé à ressembler à un véritable jeu cohérent plutôt qu'à une succession de prototypes techniques. Les différentes mécaniques commençaient enfin à fonctionner ensemble dans un environnement stable et réellement jouable.

Enfin, cette étape a également permis à l'équipe de retrouver une progression beaucoup plus fluide dans le développement. Là où la version 3D ralentissait constamment à cause de problèmes d'intégration complexes, l'architecture 2D nous permettait désormais d'ajouter rapidement de nouvelles fonctionnalités tout en conservant une stabilité globale satisfaisante du projet.

5.5) Développement des interactions et des systèmes de gameplay

Avec la reconstruction progressive du projet en 2D, l'interface utilisateur est devenue un élément de plus en plus important dans le développement de *Deadly Inspection*. Contrairement à la phase 3D, où les efforts étaient principalement concentrés sur les



problématiques techniques liées au moteur graphique et au multijoueur, la nouvelle architecture nous permettait désormais de consacrer davantage de temps à l'expérience globale du joueur.

Très rapidement, nous avons compris qu'un jeu reposant fortement sur la prise de décision et la coopération nécessite une interface claire, lisible et cohérente avec l'univers proposé. Les joueurs devaient pouvoir accéder facilement aux informations importantes sans que l'écran ne devienne surchargé ou difficile à comprendre.

Les premiers menus développés dans Pygame restaient relativement simples et fonctionnaient principalement comme des systèmes de lancement du jeu. Cependant, au fil du développement, l'interface a progressivement évolué afin de mieux s'intégrer à l'ambiance générale de *Deadly Inspection*. Nous avons commencé à travailler sur la disposition des boutons, le choix des polices d'écriture, l'organisation des fenêtres d'interaction ainsi que l'affichage des différentes informations utiles au joueur.

L'objectif principal était de créer une interface capable de renforcer l'immersion tout en restant intuitive à utiliser. Contrairement à certains jeux possédant des interfaces très complexes, nous voulions que les informations essentielles restent accessibles rapidement afin de maintenir un rythme fluide durant les phases de gameplay.

Le rôle de Bob nécessitait notamment une interface centrée sur l'analyse et l'observation. Lorsqu'un survivant se présente à l'entrée du refuge, le joueur doit pouvoir examiner rapidement différents indices sans interrompre le déroulement de la partie. Cette contrainte a fortement influencé la manière dont les fenêtres d'inspection et les systèmes d'information ont été conçus.

L'interface d'Eliott suit quant à elle une logique différente. Son gameplay étant davantage orienté vers la gestion du refuge et des ressources, ses menus nécessitent l'affichage de davantage d'informations simultanément : ressources disponibles, état du refuge, progression des recherches ou encore alertes liées à d'éventuelles crises internes.

Cette différence entre les deux interfaces contribue directement à renforcer l'asymétrie du gameplay. Même visuellement, les deux joueurs n'interagissent pas avec le jeu de la même manière, ce qui accentue la sensation de posséder des rôles réellement distincts au sein du refuge.

Parallèlement à l'interface utilisateur, un important travail a également été réalisé sur l'ambiance sonore du projet. Dès les premières phases de reconstruction en 2D, nous avons cherché à intégrer des musiques et des effets sonores capables de renforcer l'atmosphère post-apocalyptique du jeu.

Les sons d'ambiance jouent un rôle particulièrement important dans *Deadly Inspection*. Même si le style graphique reste relativement simple, l'environnement sonore permet de



transmettre une sensation de tension permanente et de fragilité du refuge. Bruits de fond, musiques mélancoliques ou effets sonores liés aux interactions contribuent progressivement à rendre l'univers du jeu plus vivant et plus immersif.

Cette attention portée à l'ambiance représente également une évolution importante dans notre manière d'aborder le projet. Durant les premières phases de développement en 3D, la majorité de nos efforts étaient concentrés sur les problématiques techniques fondamentales. Avec la reconstruction en 2D, nous pouvions enfin consacrer davantage de temps à des éléments influençant directement la qualité de l'expérience utilisateur.

Nous avons également commencé à réfléchir à différents systèmes destinés à améliorer le confort du joueur, comme l'ajout de tutoriels contextuels, l'amélioration de la lisibilité des interactions ou encore l'adaptation de l'interface à différentes résolutions d'écran. Même si certains de ces systèmes restent encore en développement, cette réflexion montre l'évolution progressive du projet vers une expérience plus complète et plus accessible.

Enfin, cette phase a marqué une transformation importante de *Deadly Inspection*. Le projet ne reposait plus uniquement sur des démonstrations techniques ou des systèmes expérimentaux, mais commençait réellement à proposer une expérience cohérente mêlant gameplay, ambiance, interface et coopération entre les joueurs.

5.6) Système d'inspection, quarantaine et Intelligence Artificielle

L'un des rôles centraux du jeu est celui de Bob, qui a la responsabilité de surveiller l'entrée du refuge et d'inspecter les survivants. Pour accomplir cette tâche, nous avons développé un système d'inspection couplé à une zone de quarantaine. Le poste de contrôle représente la principale zone d'interaction pour ce rôle. Lorsqu'un survivant se présente, le joueur utilise l'interface du scanner pour examiner différents indices physiques ou comportementaux, tels que des blessures suspectes, des traces de morsures ou une fatigue anormale.

L'interface a été conçue pour rester fluide et ne pas bloquer l'action du jeu pendant la prise de décision. L'objectif de cette mécanique n'était pas de proposer un choix évident, mais bien de générer un doute permanent. Chaque décision a des conséquences directes sur la survie du refuge : accepter la mauvaise personne peut déclencher une crise interne irréversible quelques minutes plus tard.

Lorsqu'un doute persiste, le joueur a la possibilité d'envoyer le survivant en quarantaine plutôt que de prendre une décision hâtive. Ce système place le personnage dans une cellule d'isolement pendant un temps imparti. À la fin de ce compte à rebours, le statut réel du suspect, infecté ou sain, est définitivement révélé aux joueurs, ce qui met à jour l'économie et la population de la base en conséquence. Cette mécanique de temporisation ajoute une couche



stratégique supplémentaire au gameplay, en forçant les joueurs à gérer l'espace disponible dans les cellules tout en évitant les risques immédiats de contamination.

Le cahier des charges du projet exigeait également l'intégration d'une forme d'intelligence artificielle. Au lieu de concevoir un système statique avec des règles prédéfinies, nous avons développé un algorithme capable d'apprendre de manière dynamique au cours de la partie, géré par la classe `IA_Profiler`. La phase de quarantaine constitue le véritable moteur d'apprentissage de cette IA. À la fin de la période d'isolement, lorsque la véritable nature du suspect est révélée, le jeu transmet ces informations à l'algorithme, associant les symptômes observés au résultat final.

L'IA met alors instantanément à jour ses statistiques historiques. Ainsi, lors des inspections suivantes, le scanner est capable de calculer une probabilité d'infection de plus en plus précise, affichée sous forme de pourcentage pour aider le joueur. Cette boucle crée une dynamique particulièrement intéressante : plus le joueur utilise la quarantaine intelligemment, plus l'IA devient un outil d'assistance performant, liant ainsi directement le gameplay et l'algorithmique de manière naturelle.

5.7) Système d'inspection, Quarantaine et Intelligence Artificielle

La création du mini-jeu appelé "Labo" constitue un élément central du gameplay de *Deadly Inspection*. Cette mécanique représente la victoire finale du jeu, permettant au joueur de synthétiser l'antidote contre l'infection.

Le Labo fonctionne sur trois paramètres principaux interdépendants : la progression (0 à 10 étapes), les ressources (réactifs chimiques), et la stabilité du processus. Chaque étape possède un coût progressif en réactifs. La stabilité diminue naturellement au fil du temps et est également affectée par la méthode de synthèse choisie.

Trois modes de synthèse offrent des stratégies distinctes au joueur. Le mode rapide permet une progression double mais déstabilise fortement le processus. Le mode normal propose un équilibre entre vitesse et stabilité. Le mode sécurisé progresse lentement mais reconstruit la stabilité au lieu de la perdre, créant ainsi des choix tactiques authentiques.

L'interface visuelle reproduit un écran de contrôle industriel : grille de fond animée, panneaux métalliques, LEDs clignotants, barres de progression en temps réel. Cinq boutons permettent l'interaction : exécuter la synthèse, refroidir le système, et trois pour sélectionner le mode. Des messages d'état informent le joueur des problèmes critiques (ressources insuffisantes, instabilité excessive, progression).

L'intégration du Labo dans le jeu principal a nécessité une restructuration complète du système d'événements. Le bloc gérant la touche E (interactions) était devenu complexe avec



plusieurs niveaux d'imbrication if/elif, ce qui empêchait certains cas d'être atteints. Une réorganisation hiérarchisée a permis d'ajouter le cas de l'hôpital sans affecter le scanner, la prison ou les autres interactions existantes.

La gestion des événements du Labo a été placée au début de la boucle Pygame pour capturer les événements avant d'autres systèmes. La touche ESC ferme le Labo proprement sans interférer avec la fermeture d'autres panneaux.

Concernant la victoire, le système détecte automatiquement quand la progression atteint 10/10 et définit `antidote_progress` à 100, déclenchant l'écran de victoire du jeu principal.

Pour permettre au joueur de récupérer des réactifs au cours du jeu, chaque humain sain accepté au scanner ajoute un réactif au Labo. Contrairement à la première implémentation qui réinitialisait les réactifs à 5 à chaque ouverture, les ressources persistent désormais entre les sessions. Cette modification crée une boucle de gameplay intéressante : recruter pour accumuler des réactifs, puis retourner au Labo pour progresser.

Plusieurs défis techniques ont émergé durant l'intégration. L'arborescence des événements contenait des imbrications erronées empêchant certaines conditions d'être évaluées, notamment pour l'hôpital. Un débogage systématique avec des prints a révélé que les conditions étaient correctement évaluées mais que le code imbriqué n'était jamais atteint.

Le système Labo représente une part significative de la logique finale du jeu et constitue le seul chemin vers la victoire. Il fonctionne de manière autonome et stable : ouverture via l'hôpital, persistance des données, communication avec le système principal, déclenchement automatique de la victoire. Le système n'interfère pas avec le multijoueur, le scanner ou la prison, et reste fiable même lors de parties coopératives.

Cette implémentation illustre l'importance de l'architecture logicielle dans un projet complexe et la nécessité de bien structurer le code dès les premières étapes pour éviter les problèmes d'intégration ultérieurs.

6) Développement du multijoueur et synchronisation réseau

6.1) Objectif du multijoueur

Depuis les premières phases de conception de *Deadly Inspection*, le multijoueur occupait une place centrale dans notre vision du projet. Contrairement à de nombreux jeux où le mode multijoueur constitue simplement une fonctionnalité supplémentaire, notre gameplay reposait entièrement sur la coopération entre deux joueurs possédant des rôles complémentaires. Sans communication réseau fonctionnelle, le concept même du projet perdait une grande partie de son intérêt.



Le gameplay asymétrique entre Bob et Eliott nécessitait une interaction permanente entre les joueurs. Chaque décision prise par l'un devait pouvoir avoir des conséquences visibles chez l'autre presque immédiatement. Cette dépendance imposait donc la création d'un système multijoueur capable de synchroniser correctement les déplacements, les interactions, les survivants présents dans la base ainsi que l'état général du refuge.

Dès le départ, nous savions que cette partie représenterait probablement l'un des plus grands défis techniques du projet. Aucun membre du groupe ne possédait réellement d'expérience avancée en développement réseau temps réel. Nous avons déjà étudié certaines bases théoriques liées aux sockets ou aux échanges client/serveur, mais nous n'avions encore jamais développé un véritable système multijoueur complet pour un jeu vidéo.

Malgré cela, notre ambition restait importante. Nous voulions que les deux joueurs puissent évoluer simultanément dans le même environnement, observer les actions de l'autre en temps réel et partager une expérience réellement coopérative. Cette vision semblait relativement simple en théorie, mais elle s'est rapidement révélée beaucoup plus complexe à mettre en œuvre dans la pratique.

Au début du développement, nous avons tendance à sous-estimer la difficulté réelle du multijoueur. Nous pensions principalement au fait "d'envoyer des positions" entre plusieurs joueurs. Cependant, au fil des expérimentations, nous avons progressivement découvert qu'un jeu réseau nécessite en réalité une gestion extrêmement rigoureuse des états, des synchronisations et des interactions entre les différents clients connectés.

Cette partie du projet a rapidement pris une importance énorme dans notre développement quotidien. Une grande partie de nos discussions, de nos tests et de nos séances de travail tournaient autour de problèmes de synchronisation, de décalages réseau ou de comportements incohérents entre les joueurs.

Le multijoueur est progressivement devenu à la fois l'élément le plus passionnant et le plus stressant du projet. Chaque petite réussite, comme voir enfin deux joueurs bouger ensemble correctement à l'écran, synchroniser une interaction ou réussir une connexion stable, provoquait une énorme satisfaction. À l'inverse, chaque nouveau bug réseau pouvait parfois bloquer complètement l'avancement du projet pendant plusieurs jours.

Cette alternance constante entre moments de progression et phases de blocage a fortement marqué notre expérience du développement de *Deadly Inspection*. Contrairement à d'autres parties du projet où les problèmes restaient relativement localisés, les erreurs liées au multijoueur avaient souvent des conséquences sur l'ensemble du jeu et pouvaient rapidement rendre la version complète totalement instable.

Enfin, cette importance du réseau a également profondément influencé notre manière de travailler en équipe. Le multijoueur nécessitait des phases de tests permanentes entre



plusieurs machines, des corrections fréquentes et une forte coordination entre les membres du groupe. Cette expérience nous a progressivement fait découvrir les véritables difficultés du développement coopératif temps réel dans un projet de jeu vidéo.

6.2) Premières expérimentations réseau et architecture client/serveur

Les premières expérimentations autour du multijoueur ont commencé très tôt dans le projet, dès la phase de développement en 3D sous Panda3D. À ce stade, notre objectif principal était simplement de comprendre comment plusieurs joueurs pouvaient communiquer entre eux dans un même environnement.

Nous avons choisi d'utiliser une architecture client/serveur relativement classique. Le principe était qu'un serveur central reçoive les informations envoyées par chaque joueur puis redistribue ces données aux autres clients connectés. Cette approche présentait l'avantage de centraliser les informations importantes du jeu et de limiter certains problèmes liés à la synchronisation.

Les premiers tests étaient volontairement très simples. Nous avons commencé par transmettre uniquement les positions des joueurs à travers des sockets Python. Chaque client envoyait régulièrement ses coordonnées au serveur, qui les renvoyait ensuite aux autres joueurs afin de mettre à jour leur affichage.

Même cette première étape s'est révélée plus difficile que prévu. Très rapidement, plusieurs problèmes sont apparus concernant la fréquence d'envoi des données, la stabilité des connexions et la fluidité des déplacements affichés à l'écran.

Dans certains cas, les joueurs semblaient se téléporter brusquement au lieu de se déplacer de manière fluide. Dans d'autres situations, certains mouvements n'étaient tout simplement pas transmis correctement entre les clients. Nous avons alors commencé à découvrir les premières problématiques liées à la synchronisation temps réel.

À ce stade du développement, une grande partie de notre travail reposait encore sur l'expérimentation. Nous testions différentes manières d'envoyer les données, différentes fréquences de mise à jour ainsi que plusieurs structures d'organisation du serveur.

Le développement du serveur lui-même représentait déjà une source importante de stress. Une erreur mineure dans la gestion des connexions pouvait parfois provoquer la déconnexion complète des clients ou empêcher toute communication entre les joueurs. Il arrivait régulièrement qu'une modification semblant anodine casse totalement un système qui fonctionnait quelques heures auparavant.



Cette période du projet a également marqué nos premiers véritables moments de frustration liés au développement réseau. Contrairement à des bugs classiques souvent visibles immédiatement, les problèmes réseau étaient parfois extrêmement difficiles à comprendre. Certains comportements semblaient fonctionner parfaitement sur une machine mais pas sur une autre, ou uniquement dans certaines situations précises.

Nous avons également commencé à comprendre l'importance du débogage dans le développement multijoueur. Une partie importante du travail consistait désormais à afficher des logs, suivre les paquets envoyés par le serveur, vérifier les données reçues par les clients et essayer de comprendre à quel moment précis les synchronisations échouaient.

Cette phase était parfois mentalement très fatigante. Certains bugs donnaient l'impression d'être totalement aléatoires. Il arrivait régulièrement que nous pensions avoir enfin résolu un problème avant de découvrir quelques minutes plus tard qu'il réapparaissait dans une autre situation.

Malgré ces difficultés, les premiers succès restaient extrêmement motivants. Voir plusieurs personnages apparaître simultanément dans le même environnement et réussir à déplacer deux joueurs en temps réel représentait déjà une énorme étape dans le projet. Même si le système restait encore très instable, ces premières démonstrations confirmaient que le concept multijoueur de *Deadly Inspection* pouvait réellement fonctionner.

Cette phase d'expérimentation nous a également permis de mieux comprendre les limites de notre première architecture réseau. Au fur et à mesure des tests, nous commençons déjà à voir apparaître certains problèmes qui deviendront plus importants par la suite : gestion des états du jeu, synchronisation des entités, cohérence entre les clients et difficulté d'intégration avec les autres systèmes du projet.

6.3) Reconstruction du système réseau et nouvelles approches de synchronisation

Avec l'avancement du projet et la reconstruction complète de *Deadly Inspection* en 2D sous Pygame, une grande partie du système multijoueur a dû être repensée. Même si plusieurs prototypes réseau existaient déjà durant la phase 3D, leur architecture restait difficile à intégrer proprement dans la nouvelle version du jeu.

Le passage à Pygame a représenté une opportunité importante pour reconstruire une architecture réseau plus claire et mieux adaptée aux besoins réels du gameplay. Contrairement à la phase précédente, notre objectif n'était plus simplement de "faire communiquer deux joueurs", mais de construire un véritable système capable de synchroniser correctement les différentes mécaniques du jeu dans une partie coopérative stable.



L'un des premiers changements importants concernait l'organisation générale du code réseau. Durant les premières expérimentations, une partie importante des échanges entre clients et serveur était gérée de manière relativement improvisée. Les données étaient envoyées rapidement afin de tester les connexions, mais sans véritable structure claire concernant la gestion des états du jeu.

Avec le temps, cette organisation est devenue de plus en plus difficile à maintenir. Chaque nouvelle fonctionnalité ajoutée au jeu nécessitait l'envoi de nouvelles informations réseau, ce qui augmentait progressivement la complexité du système. Les déplacements, les directions des personnages, les animations, les survivants ou encore certains événements de gameplay devaient désormais être synchronisés correctement entre les joueurs.

Nous avons alors commencé à restructurer progressivement le système réseau afin de mieux séparer les responsabilités entre le serveur et les clients. Le serveur devait désormais devenir le véritable "centre" de la partie, responsable de l'état global du jeu, tandis que les clients se concentraient principalement sur l'affichage et les interactions locales.

Cette phase a représenté un énorme travail de réflexion et de réorganisation. Une grande partie du temps de développement était désormais consacrée à revoir des systèmes déjà existants afin de les rendre plus stables et plus cohérents.

Le système de synchronisation des joueurs représentait notamment une difficulté importante. Même après plusieurs versions du serveur, certains problèmes persistaient concernant l'affichage des déplacements entre les différents clients. Dans certaines situations, un joueur pouvait voir correctement ses propres mouvements tandis que l'autre client ne recevait pas toutes les informations nécessaires à la mise à jour de sa position.

Nous avons également rencontré plusieurs problèmes liés aux états des personnages. Certaines animations ou directions de déplacement n'étaient pas toujours correctement synchronisées entre les joueurs. Il arrivait par exemple qu'un personnage continue d'apparaître en mouvement chez un autre joueur alors qu'il était déjà immobile localement.

Cette période du projet a probablement été l'une des plus stressantes techniquement. Une grande partie des bugs réseau ne produisait pas d'erreur visible dans le code lui-même, ce qui rendait leur détection particulièrement difficile. Très souvent, le jeu continuait de fonctionner "partiellement", mais avec des comportements incohérents visibles uniquement durant les tests multijoueurs.

Les phases de débogage devenaient parfois extrêmement longues. Nous passions régulièrement plusieurs heures à tester les mêmes séquences d'actions afin d'essayer de reproduire certains bugs de synchronisation. Il arrivait également qu'un problème semble totalement résolu avant de réapparaître immédiatement après une modification mineure dans une autre partie du code.



Cette situation provoquait parfois une forte fatigue mentale. Le développement réseau donnait souvent l'impression que chaque avancée pouvait casser un autre système déjà fonctionnel. Certains jours, nous avions l'impression de progresser énormément, puis de revenir presque au point de départ après l'apparition d'un nouveau problème de synchronisation.

Cependant, cette phase a également été extrêmement formatrice. Elle nous a appris l'importance de structurer correctement une architecture réseau dès le départ et de séparer clairement les responsabilités entre les différentes parties du système. Nous avons progressivement compris qu'un jeu multijoueur stable repose autant sur l'organisation du code que sur les fonctionnalités elles-mêmes.

Enfin, malgré les difficultés rencontrées, cette reconstruction progressive du système réseau a permis d'obtenir des résultats de plus en plus convaincants. Les déplacements devenaient plus fluides, les connexions plus stables et les interactions entre joueurs plus cohérentes. Même si plusieurs problèmes restaient encore présents, le projet commençait enfin à se rapprocher d'une véritable expérience coopérative jouable.

6.4) Synchronisation des survivants et gestion de la file d'attente

L'un des systèmes ayant provoqué le plus de difficultés durant le développement du multijoueur concernait la synchronisation des survivants et de la file d'attente située devant l'entrée du refuge. Cette mécanique représentait pourtant un élément central du gameplay de *Deadly Inspection*, puisque le système d'inspection repose directement sur l'arrivée progressive de survivants venant demander leur admission dans la base.

Au début du développement, nous pensions que ce système serait relativement simple à synchroniser. L'idée semblait logique : le serveur devait générer une liste de survivants puis envoyer leurs informations aux différents clients afin que tous les joueurs voient la même file d'attente. Cependant, dans la pratique, cette synchronisation s'est révélée beaucoup plus complexe que prévu.

L'un des premiers problèmes rencontrés concernait la cohérence même de la file d'attente entre les différents joueurs. Dans certaines situations, chaque client voyait une version différente des survivants présents devant la base. Un joueur pouvait voir un survivant apparaître correctement tandis que l'autre voyait un personnage différent ou parfois aucun personnage du tout.

Cette désynchronisation créait un problème majeur pour le gameplay coopératif. Comme Bob et Eliott doivent partager une expérience commune du jeu, il était essentiel que les deux



joueurs observent exactement les mêmes événements au même moment. Une différence dans la gestion de la file d'attente rendait immédiatement certaines mécaniques incohérentes.

Nous avons alors commencé à revoir entièrement la manière dont les survivants étaient générés et synchronisés. Initialement, une partie de la logique de génération existait encore côté client, ce qui augmentait fortement les risques de désynchronisation entre les différentes machines. Nous avons progressivement compris que le serveur devait devenir l'unique source de vérité concernant les survivants présents dans la partie.

Cette transition a nécessité une importante réorganisation du système. Le serveur devait désormais gérer lui-même la création des survivants, leurs positions, leur état ainsi que leur progression dans la file d'attente avant d'envoyer ces informations aux clients.

Même après cette restructuration, plusieurs problèmes continuaient d'apparaître. Dans certains cas, des survivants étaient dupliqués à l'écran ou restaient bloqués dans certaines positions. D'autres fois, les joueurs voyaient des déplacements différents pour les mêmes personnages selon le client utilisé.

Cette période a probablement représenté l'une des phases les plus éprouvantes du développement. Une grande partie du travail consistait à réaliser des tests permanents entre plusieurs machines afin d'observer précisément les différences de comportement entre les clients.

Les séances de débogage devenaient parfois particulièrement longues et frustrantes. Certains bugs semblaient totalement incompréhensibles : un survivant pouvait fonctionner parfaitement chez un joueur mais disparaître chez l'autre sans qu'aucune erreur claire n'apparaisse dans le code. Il arrivait également qu'un système fonctionne correctement pendant plusieurs tests avant de recommencer soudainement à se désynchroniser.

Le système des *RemotePlayer* représentait lui aussi une source importante de difficultés. Ces entités étaient chargées de représenter les autres joueurs à l'écran, mais plusieurs problèmes apparaissaient régulièrement concernant leurs animations, leurs déplacements ou leur état. Certaines erreurs provoquaient même des comportements incohérents où un joueur pouvait continuer d'apparaître en mouvement alors qu'il était déjà arrêté localement.

Parallèlement aux problèmes techniques, cette phase a également représenté une forte charge émotionnelle durant le développement du projet. Les longues heures de tests, les bugs récurrents et l'impression parfois de ne jamais réellement stabiliser le système provoquaient régulièrement du stress et de la fatigue au sein du groupe. Certaines séances de travail étaient entièrement consacrées à essayer de comprendre pourquoi un système qui semblait fonctionner la veille était devenu totalement instable le lendemain.

Malgré cela, chaque réussite restait extrêmement motivante. Voir enfin les deux joueurs observer la même file d'attente, réussir à synchroniser correctement les survivants ou obtenir



des déplacements cohérents représentait de véritables moments de satisfaction après plusieurs jours de débogage intensif.

Avec le recul, cette phase du projet nous a énormément appris sur les difficultés réelles du développement multijoueur temps réel. Elle nous a surtout fait comprendre que la synchronisation d'un jeu ne consiste pas uniquement à transmettre des données entre plusieurs machines, mais nécessite une gestion extrêmement précise des états, des événements et des responsabilités entre serveur et clients.

6.5) Travail collaboratif, Git et gestion des conflits technique

Le développement du système multijoueur de *Deadly Inspection* ne représentait pas uniquement un défi technique. Cette phase du projet a également profondément mis à l'épreuve notre organisation de groupe et notre manière de collaborer sur un même projet logiciel. Contrairement à des projets plus simples réalisés individuellement, le multijoueur nécessitait des modifications constantes dans des fichiers fortement liés entre eux, ce qui augmentait considérablement les risques de conflits et de problèmes d'intégration.

Très rapidement, Git est devenu un outil central dans notre manière de travailler. Chaque membre du groupe développait différentes fonctionnalités en parallèle : gestion des joueurs, synchronisation réseau, file d'attente, interactions, interfaces ou encore systèmes liés au gameplay. Cette organisation permettait d'avancer simultanément sur plusieurs parties du projet, mais elle rendait également les merges beaucoup plus complexes au fur et à mesure de l'avancement du développement.

Les premières phases de travail collaboratif semblaient relativement simples. Cependant, avec l'augmentation progressive de la complexité du projet, les conflits Git sont devenus de plus en plus fréquents. Certaines modifications réalisées sur une branche pouvaient avoir des conséquences inattendues sur d'autres systèmes développés simultanément par un autre membre du groupe.

Cette situation était particulièrement visible durant le développement du multijoueur. Le code réseau touchait directement de nombreuses parties du jeu : déplacements, joueurs distants, interactions, survivants ou encore systèmes de file d'attente. Une modification apparemment mineure pouvait parfois casser entièrement la synchronisation entre les clients.

Plusieurs périodes du projet ont ainsi été marquées par de longues phases de fusion et de correction après des merges compliqués. Il arrivait régulièrement que deux systèmes fonctionnent parfaitement séparément mais deviennent totalement incompatibles une fois réunis dans une même version du projet.



Cette partie du développement a parfois été mentalement très difficile. Après plusieurs heures passées à résoudre un problème réseau complexe, il arrivait qu'un merge Git introduise de nouveaux bugs ou rende instable une fonctionnalité précédemment corrigée. Cette impression de "casser" accidentellement des systèmes déjà fonctionnels représentait une source importante de stress.

Nous avons également progressivement appris l'importance des branches de développement dans un projet collaboratif. Certaines fonctionnalités sensibles étaient désormais développées sur des branches séparées afin de limiter les risques de casser directement la version principale du projet. Cette organisation nous permettait de tester certaines modifications plus librement avant leur intégration dans la version globale.

Malgré cela, les phases de fusion restaient parfois particulièrement compliquées. Certains conflits concernaient des portions importantes du code réseau, notamment autour de la gestion des états des joueurs ou de la synchronisation des survivants. Résoudre ces problèmes nécessitait souvent de comparer manuellement différentes versions du code afin de comprendre précisément quelles modifications devaient être conservées.

Cette période nous a également appris l'importance de la communication au sein de l'équipe. Le développement du multijoueur nécessitait des échanges permanents entre les membres du groupe afin d'éviter que plusieurs personnes modifient simultanément des systèmes incompatibles. Sans cette coordination, les risques de conflits augmentaient rapidement.

Au-delà des aspects purement techniques, cette phase a également représenté une importante expérience humaine. Certaines périodes du projet étaient particulièrement stressantes, notamment lorsque plusieurs bugs semblaient s'accumuler simultanément ou lorsqu'une fonctionnalité importante cessait soudainement de fonctionner avant une séance de test.

Il arrivait régulièrement de passer plusieurs soirées entières à tester les mêmes systèmes, corriger des bugs puis recommencer immédiatement après l'apparition d'un nouveau problème. Certaines séances de développement devenaient extrêmement fatigantes mentalement, en particulier lorsque les erreurs semblaient difficiles à reproduire ou apparaissaient de manière apparemment aléatoire.

Cependant, cette difficulté faisait également partie des moments les plus marquants du projet. Chaque bug résolu donnait une véritable impression de progression. Les moments où le système multijoueur fonctionnait enfin correctement après plusieurs jours de travail représentaient des phases particulièrement motivantes pour l'ensemble du groupe.

Avec le recul, cette expérience nous a énormément appris sur la réalité du développement collaboratif. Nous avons découvert que les difficultés humaines, organisationnelles et techniques sont souvent étroitement liées dans un projet logiciel complexe. La qualité d'un



système dépend non seulement du code lui-même, mais également de la manière dont une équipe parvient à collaborer, communiquer et s'adapter face aux problèmes rencontrés.

6.6) Ce que le développement multijoueur nous a appris

Le développement du multijoueur de *Deadly Inspection* représente probablement la partie du projet qui nous a le plus fait progresser techniquement et humainement. Même si cette phase a été marquée par de nombreuses difficultés, elle nous a permis de découvrir des problématiques extrêmement concrètes liées au développement logiciel collaboratif et à la programmation réseau temps réel.

Avant ce projet, notre vision du multijoueur restait relativement théorique. Nous avons déjà étudié certaines notions liées aux sockets ou aux échanges réseau, mais nous n'avions encore jamais été confrontés à la complexité réelle d'un système multijoueur complet intégré dans un jeu vidéo.

L'une des premières leçons importantes concerne la synchronisation des états dans un environnement temps réel. Nous avons progressivement compris qu'un jeu multijoueur ne consiste pas simplement à transmettre des positions entre plusieurs machines, mais nécessite une gestion extrêmement rigoureuse de toutes les informations partagées entre les joueurs.

Chaque élément du jeu, comme les déplacements, les survivants, les animations, les interactions ou les événements, doit être pensé en fonction de sa synchronisation réseau. Une erreur mineure dans l'organisation du système peut rapidement provoquer des incohérences importantes visibles directement par les joueurs.

Cette expérience nous a également appris l'importance de l'architecture logicielle dans un projet complexe. Au début du développement, plusieurs systèmes avaient été construits relativement rapidement afin de tester certaines idées. Cependant, au fur et à mesure de l'avancement du projet, nous avons découvert que l'organisation globale du code devient presque aussi importante que les fonctionnalités elles-mêmes.

Le développement du multijoueur nous a aussi fait prendre conscience de l'importance du débogage et des tests. Une grande partie du temps de travail n'était pas consacrée à l'ajout de nouvelles fonctionnalités, mais à comprendre précisément pourquoi certains comportements devenaient incohérents entre plusieurs clients connectés.

Cette phase nous a également appris à mieux gérer la frustration liée au développement logiciel. Certains bugs réseau pouvaient nécessiter plusieurs jours de recherche avant d'être compris correctement. Il arrivait régulièrement qu'une solution semblant fonctionner provoque ensuite de nouveaux problèmes ailleurs dans le projet.



Sur le plan humain, cette expérience nous a également permis de mieux comprendre les réalités du travail collaboratif. La communication entre les membres du groupe est devenue indispensable pour maintenir une cohérence dans le développement. Nous avons progressivement appris à mieux répartir les tâches, à organiser les branches Git et à travailler ensemble sur des systèmes fortement liés.

Enfin, cette partie du projet nous a surtout appris qu'en développement logiciel, la capacité d'adaptation est essentielle. Une grande partie du projet a consisté à revoir nos premières idées, corriger nos erreurs et reconstruire certains systèmes afin d'obtenir un résultat plus stable et plus cohérent.

Même si le développement du multijoueur a représenté l'une des parties les plus difficiles et les plus stressantes du projet, il constitue également l'expérience la plus formatrice que nous ayons rencontrée durant cette année à l'EPITA.

7.0) Immersion et sonore

7.1) Cycle jour/nuit

L'une des premières questions qu'on s'est posées en développant *Deadly Infection* était : comment donner au joueur un sentiment de progression et d'urgence sans surcharger l'interface ? Comment faire en sorte que le joueur ressente que le temps passe, que chaque décision compte, et qu'il ne joue pas dans un monde figé et sans vie ? La réponse a été d'implémenter un cycle jour/nuit complet, géré par la classe `DayNightSystem` dans le fichier `day_night.py`.

La finalité avant tout

Avant de parler du code, il faut comprendre pourquoi ce système existe. Dans un jeu de gestion et de survie comme *Deadly Infection*, l'immersion est essentielle. Si le joueur évolue dans un environnement qui ne change jamais, qui reste identique du début à la fin, il se déconnecte rapidement de l'expérience. Il joue mécaniquement, sans ressentir de pression, sans avoir l'impression que le monde autour de lui est vivant. Le cycle jour/nuit résout ce problème de plusieurs façons.

D'abord, il structure le temps de jeu. Une journée complète dure dix minutes : cinq minutes de jour, cinq minutes de nuit. Ce rythme donne au joueur des repères naturels. Il sait que quand la nuit tombe, il a tenu une demi-journée. Quand un nouveau jour commence, il a survécu à un cycle complet. Ce découpage crée inconsciemment des objectifs à court terme : "je dois tenir jusqu'à la nuit", "je dois passer cette nuit sans laisser entrer trop d'infectés". Sans ce système, le jeu serait une longue session uniforme sans points de repère.



Ensuite, il apporte une pression psychologique naturelle. Le joueur voit en permanence en haut de l'écran s'il fait jour ou nuit, ainsi que le numéro du jour en cours. Cette information simple mais visible crée une tension : combien de jours vais-je tenir ? Est-ce que je gère bien mes ressources ? Est-ce que je prends les bonnes décisions ? Le passage du temps devient lui-même un ennemi silencieux, sans qu'on ait besoin d'ajouter un compte à rebours agressif ou une alarme irritante.

Il permet aussi une mesure de performance et de rejouabilité. À la fin d'une partie, le joueur sait exactement combien de jours il a tenu. C'est une donnée simple, lisible, comparable. Elle donne envie de rejouer : "j'ai tenu 4 jours la dernière fois, est-ce que je peux faire mieux ?". C'est un mécanisme de progression implicite qui n'a pas besoin d'un système de score complexe pour fonctionner.

L'immersion visuelle : l'overlay

Au-delà du HUD affiché en haut de l'écran, le système jour/nuit agit directement sur l'apparence visuelle du jeu grâce à un overlay dynamique. Concrètement, c'est une surface pygame semi-transparente de couleur bleu foncé qui se superpose à tout ce qui est affiché à l'écran. Son opacité c'est-à-dire son niveau de transparence change automatiquement en fonction du moment de la journée.

En début de journée, l'overlay est complètement invisible : l'écran est lumineux, les couleurs sont vives. À mesure que la journée avance, quand on approche des trois quarts du temps de jour, l'overlay commence progressivement à s'assombrir. Pendant la nuit, l'opacité atteint son maximum, plongeant l'environnement dans une atmosphère sombre et pesante. Puis à l'approche de l'aube, l'overlay se dissipe progressivement, laissant revenir la lumière du jour.

Ce détail visuel, bien qu'il puisse sembler purement esthétique, a un impact réel sur le ressenti du joueur. L'assombrissement progressif crée une tension inconsciente : la nuit qui tombe signifie que le danger augmente, que la vigilance doit redoubler. C'est un signal visuel intuitif que tout le monde comprend naturellement, sans avoir besoin de l'expliquer dans un tutoriel.

La transition entre les jours

Entre chaque jour, une transition animée s'affiche : un fondu au noir progressif accompagné du texte "JOUR 2", "JOUR 3", etc. au centre de l'écran. Cette transition dure trois secondes par défaut, mais le joueur peut la sauter en appuyant sur Entrée grâce à la méthode `skip_transition()`.

Ce moment de transition a lui aussi une fonction précise. Il marque clairement la fin d'un cycle et le début d'un nouveau, comme un chapitre qui se referme et un autre qui s'ouvre. Il donne au joueur une courte respiration entre deux cycles, un instant pour souffler, évaluer sa situation, et se préparer mentalement au jour suivant. Sans cette transition, le passage d'un jour à l'autre serait imperceptible et le joueur perdrait ce sentiment de progression structurée.



En résumé

Le système jour/nuit n'est pas un simple habillage visuel. C'est un outil de game design à part entière qui structure le temps, renforce l'immersion, crée une pression naturelle et donne de la rejouabilité au jeu. Il transforme une session de jeu uniforme en une succession de cycles rythmés, chacun avec ses propres enjeux et sa propre atmosphère. C'est l'un des éléments qui fait que *Deadly Infection* se ressent comme un vrai jeu vivant, et pas simplement comme un prototype fonctionnel.

7.2) Sons d'ambiance et effet sonore

Si le système jour/nuit agit sur ce que le joueur voit, le système sonore agit sur ce qu'il ressent. Le son est l'un des éléments les plus puissants de l'immersion dans un jeu vidéo, et pourtant c'est souvent le premier qu'on néglige lors du développement. Dans *Last Gate*, nous avons fait le choix inverse : intégrer un gestionnaire sonore complet dès le début, géré par la classe SoundManager dans le fichier sound_manager.py.

La finalité avant tout

Avant de parler de la technique, il faut comprendre pourquoi le son est aussi important dans notre jeu. *Last Gate* est un jeu de tension et de décision. Le joueur doit filtrer des survivants, repérer des infectés, gérer ses ressources, tout en sentant que le temps passe et que la situation peut basculer à tout moment. Sans son, cette tension est beaucoup plus difficile à ressentir. Le cerveau humain est câblé pour associer certains sons à certaines émotions : des chants d'oiseaux évoquent la sérénité, une ambiance nocturne évoque le danger et l'inconnu, une alarme évoque l'urgence. Nous avons exploité ces associations naturelles pour renforcer l'expérience du joueur sans avoir besoin d'ajouter des éléments visuels supplémentaires.

L'objectif était donc triple : renforcer l'immersion en donnant vie à l'environnement, accompagner les mécaniques de jeu avec des retours sonores adaptés, et guider le joueur inconsciemment grâce aux sons.

L'architecture en canaux dédiés

La première décision technique importante a été d'organiser les sons en canaux séparés. Pygame permet de jouer plusieurs sons simultanément sur des canaux différents. Nous en utilisons six :

- Canal 0 : les ambiances (jour et nuit)
- Canal 1 : la musique
- Canal 2 : les alarmes
- Canal 3 : l'interface utilisateur
- Canal 4 : le scanner
- Canal 5 : les pas du joueur



Cette séparation est essentielle. Sans elle, si une alarme se déclenche au moment où le joueur se déplace, l'un des deux sons risquerait de couper l'autre. Grâce à l'utilisation de canaux dédiés, tous les sons peuvent coexister sans interférer entre eux. Le joueur peut ainsi entendre simultanément l'ambiance nocturne, ses propres pas et une alarme, chacun disposant de son propre volume et de son propre canal audio.

Le crossfade jour/nuit : le cœur du système

La fonctionnalité la plus élaborée du SoundManager est le crossfade entre les ambiances jour et nuit. Plutôt que de couper brutalement la musique du jour et de démarrer celle de la nuit, les deux sons se fondent progressivement l'un dans l'autre sur une durée de trois secondes.

Concrètement, lorsque le système détecte un changement d'état, comme le passage du jour à la nuit, il démarre la nouvelle ambiance sonore avec un volume nul. À chaque frame, le volume du nouveau son augmente progressivement tandis que celui de l'ancienne ambiance diminue, jusqu'à ce que la transition soit totalement terminée. Ce fondu enchaîné reste pratiquement imperceptible pour le joueur : il ne remarque pas directement le changement de son, mais ressent naturellement l'évolution de l'atmosphère. C'est précisément l'effet recherché.

Cette technique s'articule directement avec le système jour/nuit : c'est la classe DayNightSystem qui signale au SoundManager quand basculer, créant ainsi une synchronisation parfaite entre ce que le joueur voit et ce qu'il entend.

Les ambiances : donner vie à l'environnement

Nous avons choisi deux ambiances sonores distinctes pour le jour et la nuit. Le jour est accompagné d'un son de nature avec des chants d'oiseaux, évoquant une relative sécurité, un monde encore vivant malgré l'apocalypse. La nuit, au contraire, est accompagnée d'une ambiance nocturne plus pesante, avec des sons plus graves et plus inquiétants.

Ce contraste sonore renforce également le contraste visuel de l'overlay. Le joueur ne se contente pas de voir l'écran s'assombrir, il perçoit aussi l'atmosphère évoluer à travers l'ambiance sonore. Les deux systèmes fonctionnent ainsi ensemble afin de créer une expérience cohérente et immersive. C'est cette combinaison qui permet au jeu de dépasser le simple aspect visuel pour véritablement transmettre une sensation au joueur.

7.3) Le menu principale

Le menu est la première chose que le joueur voit en lançant *Last Gate*. C'est la vitrine du jeu, et elle compte énormément : un menu soigné donne immédiatement envie de jouer, il pose l'ambiance et l'univers avant même que la première mécanique soit montrée.



Techniquement, c'est une image PNG qui prend tout l'écran, sur laquelle on a placé des zones cliquables invisibles positionnées manuellement par-dessus les boutons de l'image. Cette approche nous a permis de concevoir librement l'apparence du menu sans être limités par les composants visuels de pygame.

Dès l'ouverture, une musique se lance automatiquement. Ce détail transforme l'attente devant le menu en une vraie expérience, et crée une transition naturelle vers le jeu.

Au départ du projet, le bouton Options existait mais n'était connecté à aucune fonctionnalité. J'ai développé un panneau de paramètres comprenant trois sliders, dédiés au volume de la musique, au volume des effets sonores et à la luminosité, entièrement réalisés à la main avec une mise à jour en temps réel. Donner ce niveau de contrôle au joueur constitue une marque de finition importante, car un jeu dépourvu d'options sonores peut rapidement donner l'impression d'être inachevé.

Enfin, j'ai ajouté le menu multijoueur, qui permet d'héberger une partie ou d'en rejoindre une via une adresse IP. C'est le point d'entrée vers la dimension coopérative du jeu.

8.0) Améliorations

Suite au développement de notre projet, plusieurs axes d'amélioration ont été identifiés afin de perfectionner l'expérience utilisateur et enrichir le contenu du jeu.

Amélioration du système de scanner :

Le scanner actuel manque de précision et peut s'avérer difficile à prendre en main pour un nouvel utilisateur. Il serait donc nécessaire de le rendre plus intuitif et plus fiable, notamment en ajoutant des indications claires sur son fonctionnement.

Amélioration de l'interface du laboratoire :

L'interface du laboratoire, qui représente la surface de jeu, n'est pas suffisamment explicite. Sans explication préalable, elle peut être difficile à comprendre. Une refonte visuelle accompagnée d'indications contextuelles permettrait une meilleure prise en main.

Amélioration de l'ambiance sonore :

L'atmosphère du jeu gagnerait à être enrichie par l'ajout de musiques et d'effets sonores supplémentaires, afin de renforcer l'immersion du joueur.

Amélioration du gameplay :

Le gameplay offre de nombreuses possibilités d'évolution:



- Ajouter un bouton dédié pour combattre les zombies, le système actuel étant trop peu réactif lorsqu'on se fait attaquer ;
- Augmenter le nombre de zombies et les rendre plus résistants pour accroître la difficulté ;
- Complexifier la gestion des vies du joueur ;
- Introduire des mécaniques d'autosuffisance, comme des vagues nocturnes où les zombies attaquent la base et où le joueur doit la défendre ;
- Mettre en place un système d'armes et de construction de barrières.

Amélioration du système de ressources :

Actuellement, les améliorations de la base reposent uniquement sur l'argent. Il serait plus immersif et stratégique d'introduire un système de ressources variées, comme le bois ou la ferraille, afin d'ajouter une dimension de gestion et de survie plus réaliste.

9.0) Conclusion

Ce projet a représenté un véritable défi pour notre équipe, mais il nous a apporté énormément, tant sur le plan technique qu'humain.

Nous avons découvert que le travail en équipe n'est pas une chose simple à gérer. Nous avons traversé des moments difficiles, fait face à des désaccords et dû résoudre de nombreux problèmes en cours de route. Pourtant, malgré toutes ces tensions, nous avons toujours réussi à nous accorder et à avancer ensemble. Cette expérience nous a permis de comprendre que le vrai travail d'équipe, celui qui reflète ce que l'on retrouve dans le monde professionnel, s'apprend avec le temps. Il demande de la patience, de l'ouverture d'esprit et la capacité à prendre du recul pour que le collectif prime sur l'individuel. En ce sens, ce projet nous a apporté une vraie maturité et un regard différent sur la collaboration.

Au-delà de la dimension humaine, nous avons également appris à développer un véritable jeu vidéo, ce qui n'est pas une mince affaire. Nous avons notamment dû effectuer un changement de cap important en passant de la 3D à la 2D, ce qui n'était pas évident, surtout dans un délai relativement court. Et pourtant, nous avons réussi à mettre en place l'essentiel de ce que nous avions initialement prévu, ce qui est en soi une vraie satisfaction.

Oui, nous avons connu des échecs. Oui, nous avons manqué de temps et rencontré des difficultés. Mais au final, nous avons abouti à un jeu jouable, agréable et appréciable. Il n'est peut-être pas exceptionnel, mais pour un premier jeu vidéo réalisé en équipe, nous pensons sincèrement que nous pouvons en être fiers.